

PROJECT REPORT ON
"SKILLFLOW - SKILL SWAPPING AND LEARNING PLATFORM"

Submitted in partial fulfillment of the requirement for the award of the degree

BACHELOR OF COMPUTER APPLICATION



Session: 2025-2026

Project Guided Name

Internal Guide:

Prof. Sudhir Kumar Solanki

Study Institute:

Lokmanya Tilak Science
& Commerce College

Study Institute Code: 7351

Submitted By:

Nazia Khan

Albira khan

Anjum Khan

Enrollment No.

AZ1437315038

AZ1437315108

AZ1437315012

Roll No.

212436

212428

212432

Submitted To:

Makhanlal Chaturvedi National University of Journalism and Communication, Bhopal

PROJECT REPORT ON "SKILLFLOW - SKILL SWAPPING AND LEARNING PLATFORM"

Submitted in partial fulfillment of the requirement for the award of the degree

BACHELOR OF COMPUTER APPLICATION



Session: 2025-2026

Project Guided Name

Internal Guide:

Prof. Sudhir Kumar Solanki

Study Institute:

Lokmanya Tilak Science
& Commerce College

Study Institute Code: 7351

Submitted By:

Nazia Khan

Albira khan

Anjum Khan

Enrollment No.

AZ1437315038

AZ1437315108

AZ1437315012

Roll No.

212436

212428

212432

Submitted To:

Makhanlal Chaturvedi National University of Journalism and Communication, Bhopal

PROJECT CERTIFICATE

This is to certify that the project report entitled **SKILLFLOW - SKILL SWAPPING AND LEARNING PLATFORM** has been submitted to Makhanlal Chaturvedi National

University of Journalism & Communication, in partial fulfillment of the requirement for the award of the degree of Bachelor of Computer Applications(BCA), is original work carried out by myself Nazia Khan, Albira Khan and Anjum Khan with enrolment no. AZ1437315038, AZ1437315108 and AZ1437315012 under the Supervision of Prof./Dr./Mr./Ms._____.

The matter embodied in this project is genuine work done by myself and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Date:

Name :

Nazia Khan

Albira Khan

Anjum Khan

Contact Details:

Khannazia2507@gmail.com

alvirakhan149@gmail.com

Khananjum82250@gmail.com

Verified by the Supervisor

Name & Signature of the

Supervisor/s

Date:

**MAKHANLAL CHATURVEDI NATIONAL UNIVERSITY OF JOURNALISM &
COMMUNICATION, BHOPAL**

PROJECT WORK EVALUATION

1. Candidate's Roll No.: 212436, 212428, 212432

Name: Nazia Khan, Albira Khan, Anjum Khan

E-mail: Khannazia2507@gmail.com, alvirakhan149@gmail.com, Khananjum82250@gmail.com

2. Project Title: SKILLFLOW - SKILL SWAPPING AND LEARNING PLATFORM

3. Software Base: Python, MySQL (XAMPP Server)

4. Sponsoring Institute Address: Lokmanya Tilak Science and Commerce College, Ujjain

5. Submitted for Course & Year: Bachelor of Computer Application (2021–2024)

6. Study Centre Code, Name, Address: 7315, Lokmanya Tilak Science and Commerce College, Ujjain

7. Total Internal Assessment and Exam Work Evaluation: Marks out of _____

Details of evaluation along with name and sign of teacher

Particulars	Marks out of	Marks Awarded	Internal Name & Signature	Examiner's

8. Forwarding by Head of Study Center:
(Signature, Name & Study Centre Seal)

9. Remarks of Study Center: _____

10. University Project Examiner's Evaluation
Total Marks awarded _____ out of _____

Details Evaluation along with name & sign of University Examiner

Particulars	Marks out of	Marks Awarded	External Name & Signature	Examiner's

11. Remarks of University Examiner: _____

CERTIFICATE FROM PROJECT GUIDE

This is certify that this Major Project entitled "**SKILLFLOW - SKILL SWAPPING AND LEARNING PLATFORM**" submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Application (BCA) in session (years 2025 to 2026) to the Makhanlal Chaturvedi National University of Journalism & Communication, Bhopal, done by

Nazia Khan, Albira Khan & Anjum Khan is an authentic work carried out by his/her them at "Lokmanya Tilak Science and Commerce College Ujjain under my guidance. The matter and software embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of BCA Teacher

(Project Guide)

SELF DECLARATION

This is to certify that the Major Project report entitled " **SKILLFLOW - SKILL SWAPPING AND LEARNING PLATFORM** " is done by me, and it is authentic work carried out for the partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Application (BCA) under the guidance The matter and software embodies in this project has not been submitted earlier for award of any degree or diploma to the best of my knowledge and believes.

Date:

Signature of Student

Nazia Khan

Roll No:

212436

Enrollment No:

AZ1437315038

SELF DECLARATION

This is to certify that the Major Project report entitled " **SKILLFLOW - SKILL SWAPPING AND LEARNING PLATFORM** " is done by me, and it is authentic work carried out for the partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Application (BCA) under the guidance The matter and software embodies in this project has not been submitted earlier for award of any degree or diploma to the best of my knowledge and believes.

Date:

Signature of Student

Albira Khan

Roll No:

212428

Enrollment No:

AZ1437315108

SELF DECLARATION

This is to certify that the Major Project report entitled " **SKILLFLOW - SKILL SWAPPING AND LEARNING PLATFORM** " is done by me, and it is authentic work carried out for the partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Application (BCA) under the guidance The matter and software embodies in this project has not been submitted earlier for award of any degree or diploma to the best of my knowledge and believes.

Date:

Signature of Student

Anjum Khan

Roll No:

212432

Enrollment No:

AZ1437315102

ACKNOWLEDGEMENT

This Major Project is the result of contributions of many minds. I would like to acknowledge and thank my project guide Sudhir Kumar Solanki. (Faculty Name) for his/her valuable support and guidance. He/she guided me through the process from conception and till the completion of this project. I would also like to thank my class teacher/institute directorand my all my faculties.....I thank to lab staff members.....and other non-teaching members. I am very thankful for the open-handed support extended by many people. While no list would be complete, it is my pleasure to acknowledge the assistance of my friends who provided encouragement, knowledge, and constructive suggestions.

Signature of Student

Nazia Khan

Albira Khan

Anjum Khan

Roll No

212436

212428

212432

Enrollment No

AZ1437315038

AZ1437315108

AZ1437315012

TABLE OF CONTENTS

INDEX / TABLE OF CONTENTS

Chapter No.	Chapter / Section Title	Page No.
Preliminary	Cover Page, Certificate, Declaration, Acknowledgement and Abstract	i-v
1	Introduction: project overview, skill exchange concept, need and scope	1
2	Existing System: limitations of manual/offline skill exchange	24
3	Proposed System: SkillFlow workflow, modules and benefits	38
4	Feasibility Study: technical, economic and operational feasibility	55
5	Requirement Analysis: software, hardware, users and functional needs	69
6	System Design: architecture, DFD, ER diagram, UML and database design	88
7	Module Explanation: authentication, OTP, dashboard, matching and admin panel	124
8	Coding and Implementation: selected SkillFlow source code with explanation	162
9	Validation and Security: form checks, sessions, roles and protected workflows	218
10	Testing: functional testing, UI testing, database testing and test cases	236
11	Results and Screenshots: output pages and screenshot placeholders	258
12	Advantages and Limitations: project benefits and practical constraints	286
13	Future Scope: AI matching, mobile application and scalability	294
14	Conclusion: final outcome and objectives achieved	300
15	Bibliography and References	303
16	Appendix: supporting material and project notes	305

LIST OF FIGURES

Figure No.	Figure Title	Chapter
Figure 5.1	System Architecture	System Design
Figure 5.2	DFD Level 0	System Design
Figure 5.3	DFD Level 1	System Design
Figure 5.4	ER Diagram	System Design
Figure 5.5	Use Case Diagram	System Design
Figure 5.6	Activity Diagram	System Design
Figure 5.7	User Chat Unlock Flowchart	System Design
Figure 5.8	Admin Management Flowchart	System Design
Figure 5.9	Payment Flow	System Design
Figure 5.10	Deployment Architecture	System Design
Figure 5.11	Database Relationship	System Design

LIST OF TABLES

Table No.	Table Title	Chapter
Table 6.1	admin_login_activity	Database Design
Table 6.2	admin_notifications	Database Design
Table 6.3	Reports	Database Design
Table 6.4	admin_settings	Database Design
Table 6.5	admin_accounts	Database Design
Table 6.6	skill_categories	Database Design
Table 6.7	user_profiles	Database Design
Table 6.8	skill_categories	Database Design
Table 6.9	user_notifications	Database Design
Table 6.10	user_activity	Database Design
Table 6.11	admin_settings	Database Design
Table 6.12	Matches	Database Design
Table 6.13	user_reviews	Database Design
Table 6.14	user_favorites	Database Design
Table 6.15	user_notifications	Database Design
Table 6.16	Notifications	Database Design
Table 6.17	Messages	Database Design

Table 6.18	unfollow_reports	Database Design
Table 6.19	Requests	Database Design
Table 6.20	Payments	Database Design
Table 6.21	user_skill_categories	Database Design
Table 6.22	user_reviews	Database Design
Table 6.23	Skills	Database Design
Table 6.24	Reviews	Database Design
Table 6.25	user_notifications	Database Design
Table 6.26	user_settings	Database Design
Table 6.27	Users	Database Design
Table 6.28	subskill_categories	Database Design
Table 6.29	Reports	Database Design
Table 6.30	admin_actions	Database Design

LIST OF CODES / CODING INDEX

Code Number	Code Title	Module Name	Page Number
Code 9.1	Database Connection	Database	Update in Word
Code 9.2	User Registration	Authentication	Update in Word
Code 9.3	User Login	Authentication	Update in Word
Code 9.4	OTP Verification	Security	Update in Word
Code 9.5	Password Reset	Authentication	Update in Word
Code 9.6	Skill Listing	Skill Discovery	Update in Word
Code 9.7	Skill Details	Skill Discovery	Update in Word
Code 9.8	Search and Filter	Search	Update in Word
Code 9.9	Saved Profile Logic	Saved Profile	Update in Word
Code 9.10	Add to Request List	Request List	Update in Word
Code 9.11	Chat Unlock	Chat Unlock	Update in Word
Code 9.12	Request Placement	Requests	Update in Word

Code 9.13	Payment Processing	Payment	Update in Word
Code 9.14	Payment Success Verification	Payment	Update in Word
Code 9.15	Request Tracking	Requests	Update in Word
Code 9.16	Admin Login	Admin	Update in Word
Code 9.17	Add Skill	Skill Management	Update in Word
Code 9.18	Edit Skill	Skill Management	Update in Word
Code 9.19	Delete Skill	Skill Management	Update in Word
Code 9.20	Skill Category Management	Skill Discovery	Update in Word
Code 9.21	Request Management	Admin Requests	Update in Word
Code 9.22	Payment Management	Admin Payment	Update in Word
Code 9.23	Reports Analytics	Reports	Update in Word
Code 9.24	Notification System	Notifications	Update in Word
Code 9.25	Activity Logs	Security Audit	Update in Word

LIST OF ABBREVIATIONS

Abbreviation	Full Form	Usage
BCA	Bachelor of Computer Application	Academic programme
DBMS	Database Management System	MySQL storage
ERD	Entity Relationship Diagram	Database design
DFD	Data Flow Diagram	System design
OTP	One Time Password	Verification
CRUD	Create Read Update Delete	Admin operations
SQL	Structured Query Language	Database queries
XSS	Cross Site Scripting	Security

CHAPTER 1 - INTRODUCTION

Introduction to Skill Exchange

Introduction to Skill Exchange in SkillFlow introduces how the platform converts informal peer learning into an organized digital workflow where users can publish skills, discover compatible learners and begin structured exchanges through a Flask powered interface.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
User usability	User usability is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Admin control	Admin control is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Database accuracy	Database accuracy is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Security	Security is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Skill Swapping Concept

Skill Swapping Concept in SkillFlow explains the learning exchange idea from the user's point of view, showing how a person moves from registration to skill discovery, request creation, matching and controlled communication.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Need of Online Skill Exchange

Need of Online Skill Exchange in SkillFlow focuses on the practical need for a web based environment where students and professionals can trade knowledge without depending on offline contacts or scattered social media messages.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

About SkillFlow

About SkillFlow in SkillFlow describes the project's core purpose: to bring skill listing, matching, request handling, chat access, notifications and administrative supervision into one connected application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Objectives of the Project

Objectives of the Project in SkillFlow presents the academic and technical goals of the project, especially the use of Python Flask, HTML, CSS, JavaScript and MySQL to solve a real collaboration problem.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Scope of the Project

Scope of the Project in SkillFlow defines the boundary of the application by separating learner features, admin controls, database operations, payment supported access and report handling into clear functional areas.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Problem Statement

Problem Statement in SkillFlow discusses the problem that learners often know what they want to learn but do not have a reliable way to find someone who needs their own skill in return.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

SDLC Model Used

SDLC Model Used in SkillFlow shows how the development process was planned around requirement study, database design, route implementation, interface preparation, validation, testing and final deployment readiness.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Importance of Admin Automation

Importance of Admin Automation in SkillFlow explains how administrative automation supports the project by monitoring users, reports, requests, payments and platform activity from a controlled backend panel.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Importance of Database Driven Website

Importance of Database Driven Website in SkillFlow highlights the database driven nature of the system, where important actions are stored, retrieved and updated through MySQL tables instead of remaining temporary browser data.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Growth of Digital learning Systems

Growth of Digital learning Systems in SkillFlow connects digital learning requirements with practical software engineering by combining authentication, search, matching, messaging and analytics into a single student project.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

User Convenience and Business Value

User Convenience and Business Value in SkillFlow emphasizes user convenience through clean navigation, responsive pages, visible status messages, quick notifications and a dashboard that keeps major actions easy to reach.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

CHAPTER 2 - SYSTEM ANALYSIS

Existing Offline Skill Business

Existing Offline Skill Business in SkillFlow studies the existing limitations of manual learning exchange and explains why a dedicated platform gives better tracking, accountability and communication control.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
User usability	User usability is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Admin control	Admin control is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Database accuracy	Database accuracy is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Security	Security is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Problems in Manual Skill Enquiry

Problems in Manual Skill Enquiry in SkillFlow presents the proposed solution as a manageable web application where every major process is represented through clear routes, templates, forms and database relationships.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

User Problems

User Problems in SkillFlow examines technical feasibility through the availability of Flask, MySQL, browser based frontend tools and a development environment that can be maintained on standard systems.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Inventory Problems

Inventory Problems in SkillFlow evaluates operational feasibility by considering whether normal users can register, add skills, send requests, unlock chat and understand notifications without technical training.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Manual Request Problems

Manual Request Problems in SkillFlow discusses economic feasibility because the project can be built with open source technologies while still supporting premium style features such as plans and payment records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Payment Problems

Payment Problems in SkillFlow organizes requirement analysis around real users, admin responsibilities, security checks, database fields and the communication flow required for learning exchanges.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Proposed SkillFlow System

Proposed SkillFlow System in SkillFlow presents system design as the bridge between project idea and implementation, converting modules into architecture, diagrams, tables, pages and backend workflows.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Benefits of Proposed System

Benefits of Proposed System in SkillFlow describes the architecture through the browser, Flask application layer, route controllers, MySQL database and admin modules that cooperate during every transaction.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Comparative Analysis

Comparative Analysis in SkillFlow explains data movement by showing how inputs from forms become requests, matches, messages, notifications, reviews, reports and analytics records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

CHAPTER 3 - FEASIBILITY STUDY

Technical Feasibility

Technical Feasibility in SkillFlow represents entity relationships among users, skills, requests, matches, payments, messages and reports so that database dependency can be understood clearly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Economic Feasibility

Economic Feasibility in SkillFlow documents user behaviour through UML style views that make login, profile editing, request sending, chat access and admin review easier to explain during viva.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Operational Feasibility

Operational Feasibility in SkillFlow describes database design with attention to field purpose, relationship meaning, integrity and the way MySQL supports persistent project records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Legal Feasibility

Legal Feasibility in SkillFlow explains each module by connecting the page seen by the user with the route, validation checks and table operations performed in the backend.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Schedule Feasibility

Schedule Feasibility in SkillFlow presents authentication as the first controlled entry point where user identity, session values and validation rules protect the rest of the application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Risk Feasibility

Risk Feasibility in SkillFlow explains OTP verification as an additional confirmation step that improves trust before sensitive account actions are accepted by the system.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Resource Feasibility

Resource Feasibility in SkillFlow studies skill matching as the main business logic where offered skills, required skills and user interests are compared to suggest meaningful exchange opportunities.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

CHAPTER 4 - SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

Software Requirements in SkillFlow explains the chat system as a controlled communication area that becomes useful after the system has enough information to connect two users responsibly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
User usability	User usability is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Admin control	Admin control is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Database accuracy	Database accuracy is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Security	Security is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Hardware Requirements

Hardware Requirements in SkillFlow discusses payment integration through plan selection, transaction recording and administrative visibility rather than treating payment as a separate isolated feature.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Development Tools

Development Tools in SkillFlow describes the dashboard as the user's working area where personal details, skills, requests, matches, notifications and recent activity are brought together.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Browser Requirements

Browser Requirements in SkillFlow presents the admin panel as the management side of the software where platform activity can be supervised through users, reports, plans, analytics and notifications.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Server Requirements

Server Requirements in SkillFlow explains validation by showing how forms, route checks, database constraints and user feedback reduce incorrect entries and incomplete submissions.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Database Requirements

Database Requirements in SkillFlow discusses security from the viewpoint of sessions, password handling, OTP checks, role separation, controlled routes and careful processing of user supplied values.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Network and Deployment Requirements

Network and Deployment Requirements in SkillFlow explains testing through realistic operations such as registration, login, skill posting, request creation, chat unlock, payment recording and admin review.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

CHAPTER 5 - SYSTEM DESIGN

System Architecture Diagram

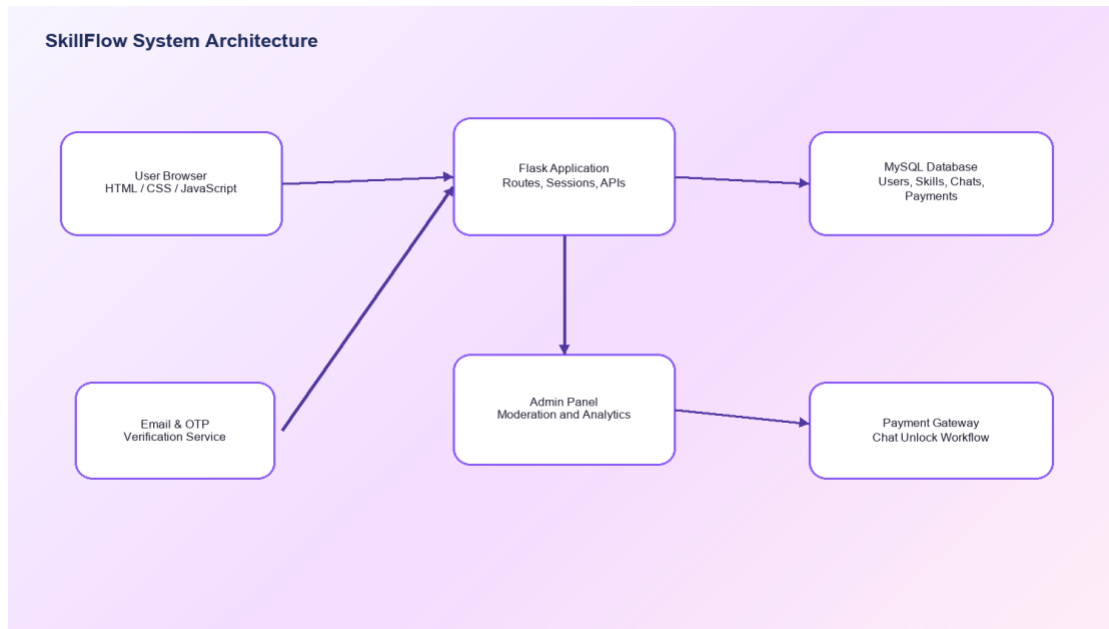


Figure: System Architecture Diagram. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

System Architecture Diagram Explanation

System Architecture Diagram Explanation in SkillFlow presents results by connecting completed features with observable outputs on pages, database records and admin dashboard summaries.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

DFD Level 0

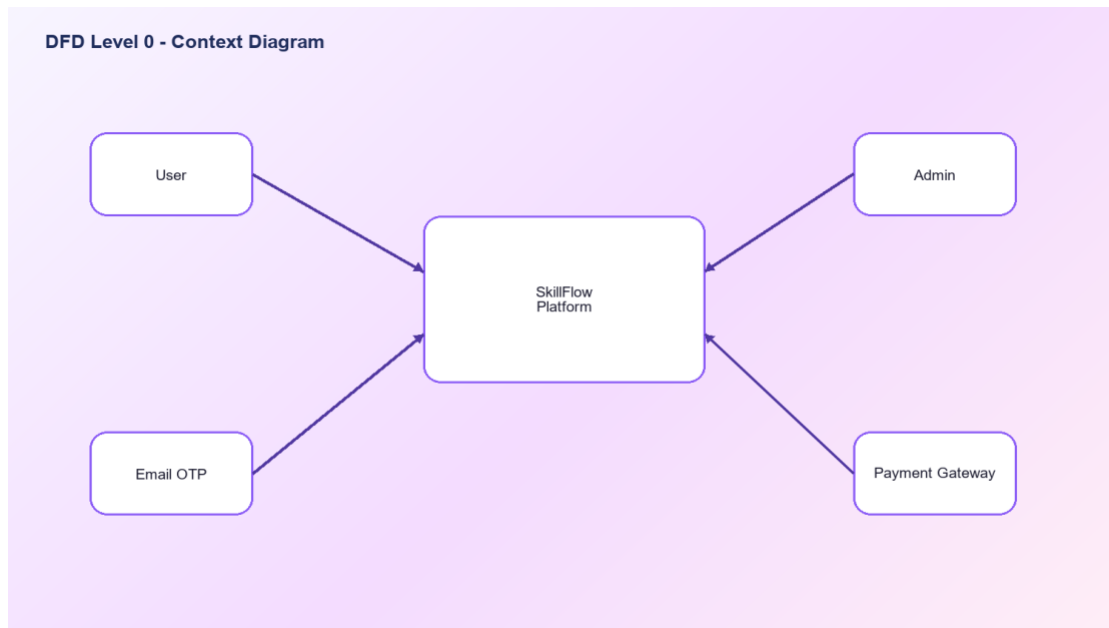


Figure: DFD Level 0. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

DFD Level 0 Explanation

DFD Level 0 Explanation in SkillFlow describes advantages through practical benefits such as organized exchange, reduced manual search, better accountability and improved learning collaboration.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

DFD Level 1

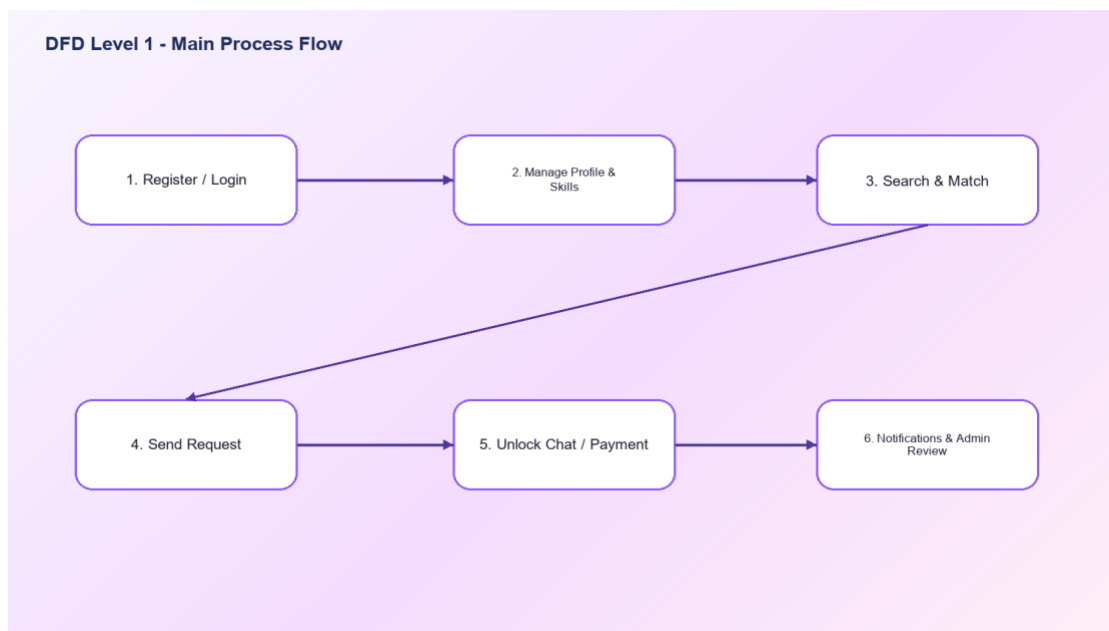


Figure: DFD Level 1. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

DFD Level 1 Explanation

DFD Level 1 Explanation in SkillFlow discusses limitations honestly, including the dependency on correct user data, internet availability and future scope for stronger recommendation intelligence.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

ER Diagram

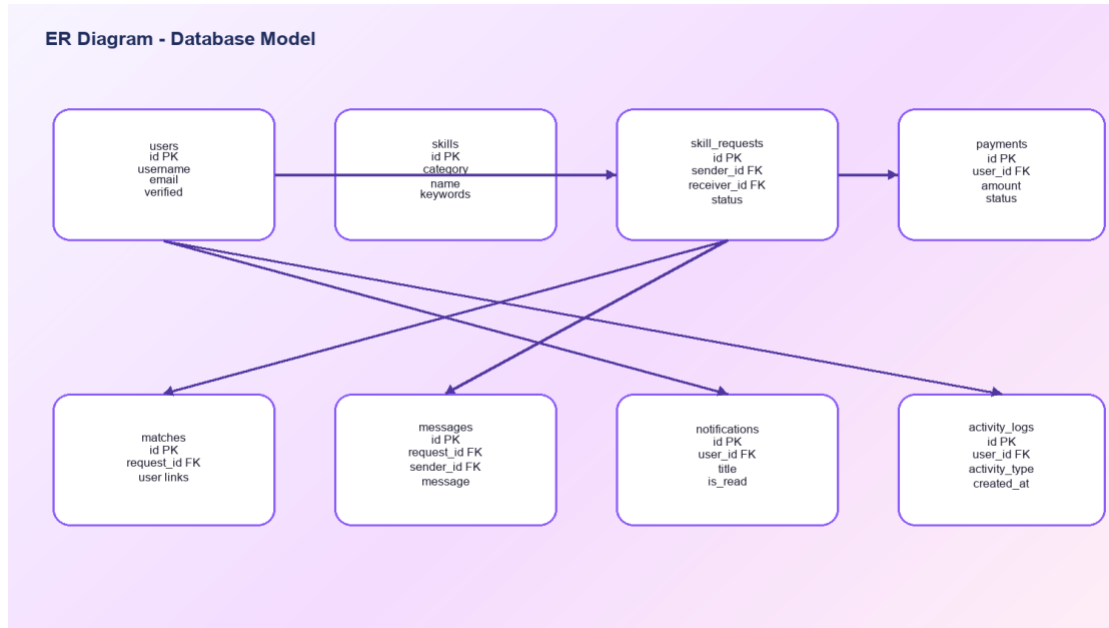


Figure: ER Diagram. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

ER Diagram Explanation

ER Diagram Explanation in SkillFlow presents future scope by identifying improvements such as AI matching, mobile application support, advanced analytics, richer communication and scalable deployment.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Use Case Diagram

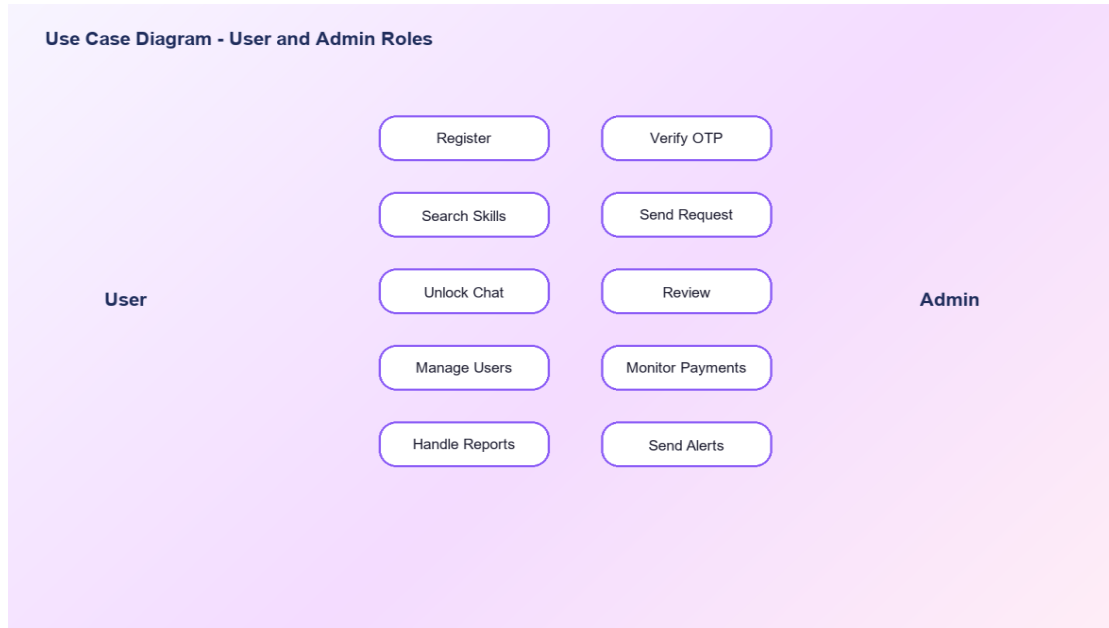


Figure: Use Case Diagram. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

Use Case Diagram Explanation

Use Case Diagram Explanation in SkillFlow concludes the project by summarizing how the planned learning exchange system was converted into a working Flask and MySQL based web application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Activity Diagram



Figure: Activity Diagram. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

Activity Diagram Explanation

Activity Diagram Explanation in SkillFlow introduces how the platform converts informal peer learning into an organized digital workflow where users can publish skills, discover compatible learners and begin structured exchanges through a Flask powered interface.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

User Chat Unlock Flowchart

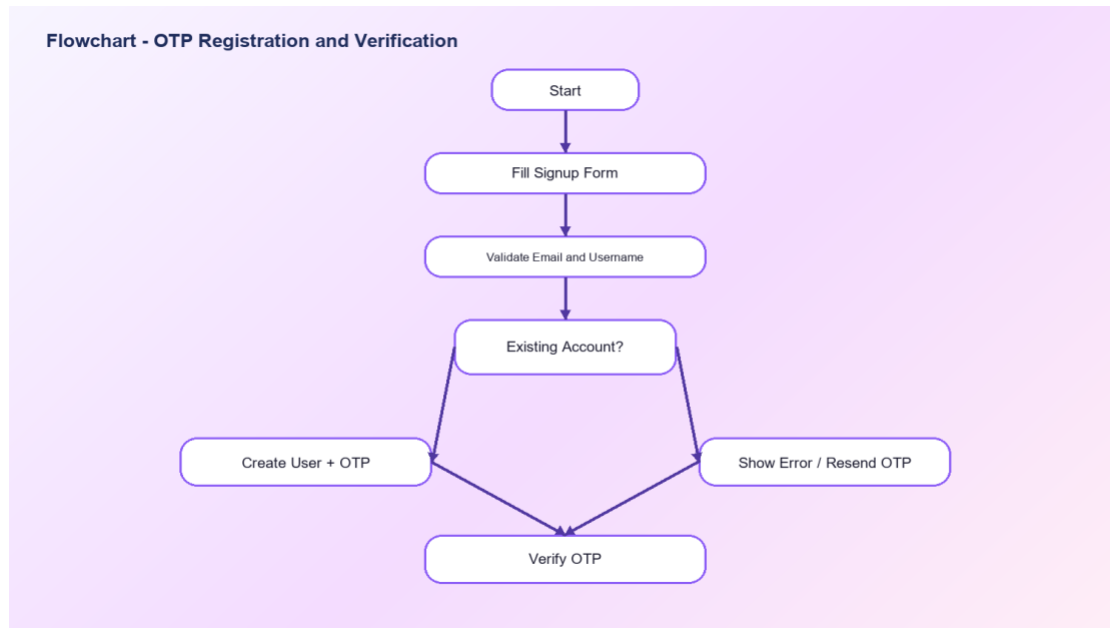


Figure: User Chat Unlock Flowchart. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

User Chat Unlock Flowchart Explanation

User Chat Unlock Flowchart Explanation in SkillFlow explains the learning exchange idea from the user's point of view, showing how a person moves from registration to skill discovery, request creation, matching and controlled communication.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Admin Management Flowchart

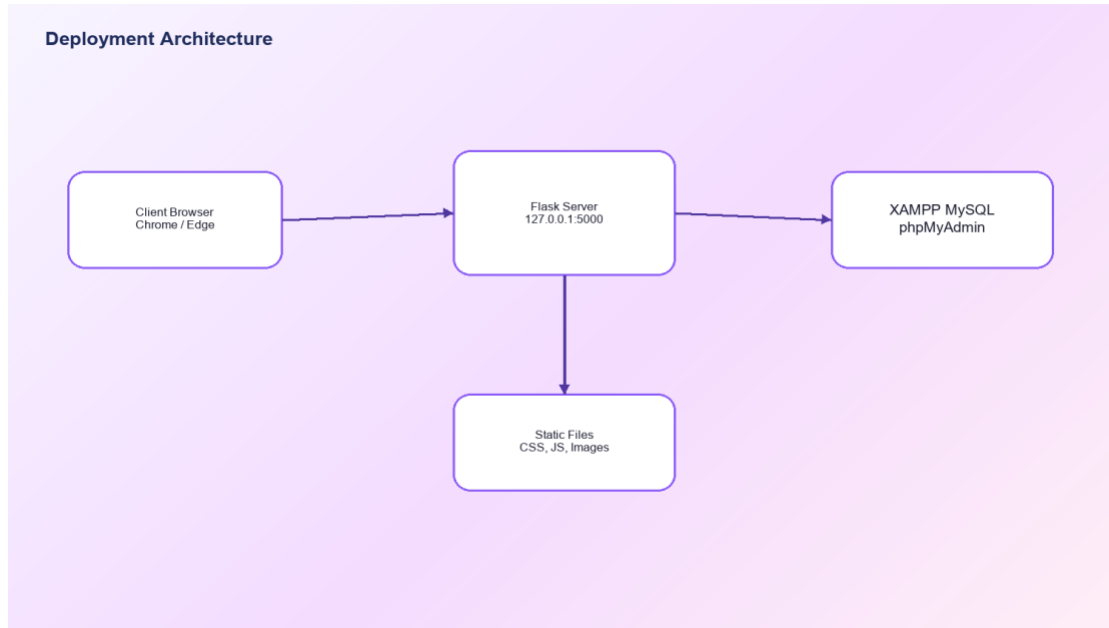


Figure: Admin Management Flowchart. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

Admin Management Flowchart Explanation

Admin Management Flowchart Explanation in SkillFlow focuses on the practical need for a web based environment where students and professionals can trade knowledge without depending on offline contacts or scattered social media messages.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Payment Flow Diagram

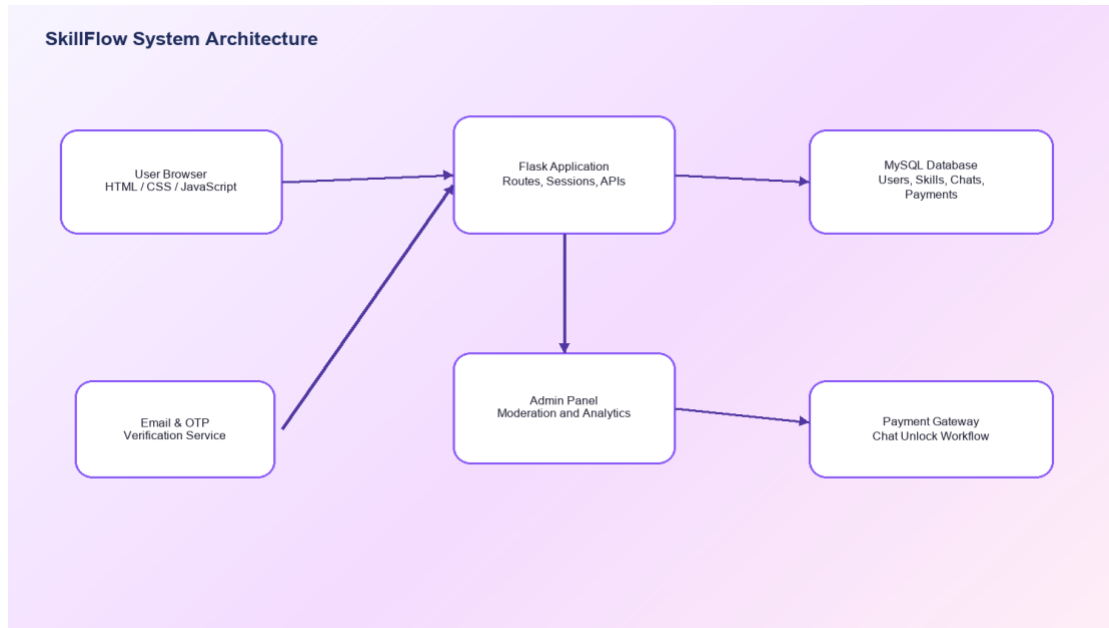


Figure: Payment Flow Diagram. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

Payment Flow Diagram Explanation

Payment Flow Diagram Explanation in SkillFlow describes the project's core purpose: to bring skill listing, matching, request handling, chat access, notifications and administrative supervision into one connected application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Deployment Architecture

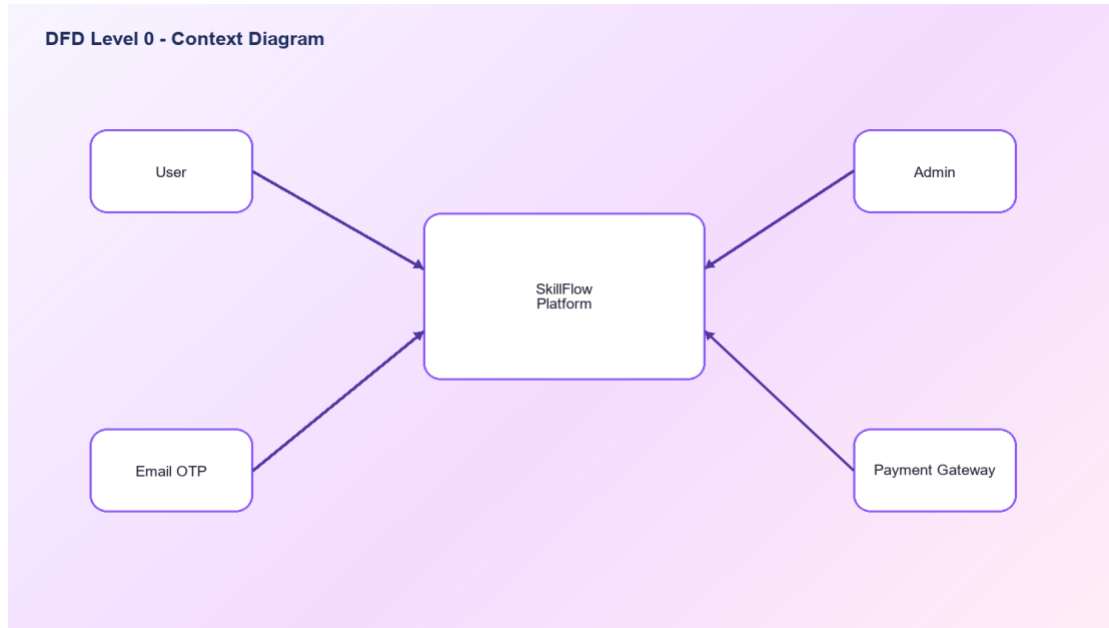


Figure: Deployment Architecture. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

Deployment Architecture Explanation

Deployment Architecture Explanation in SkillFlow presents the academic and technical goals of the project, especially the use of Python Flask, HTML, CSS, JavaScript and MySQL to solve a real collaboration problem.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Database Relationship Diagram

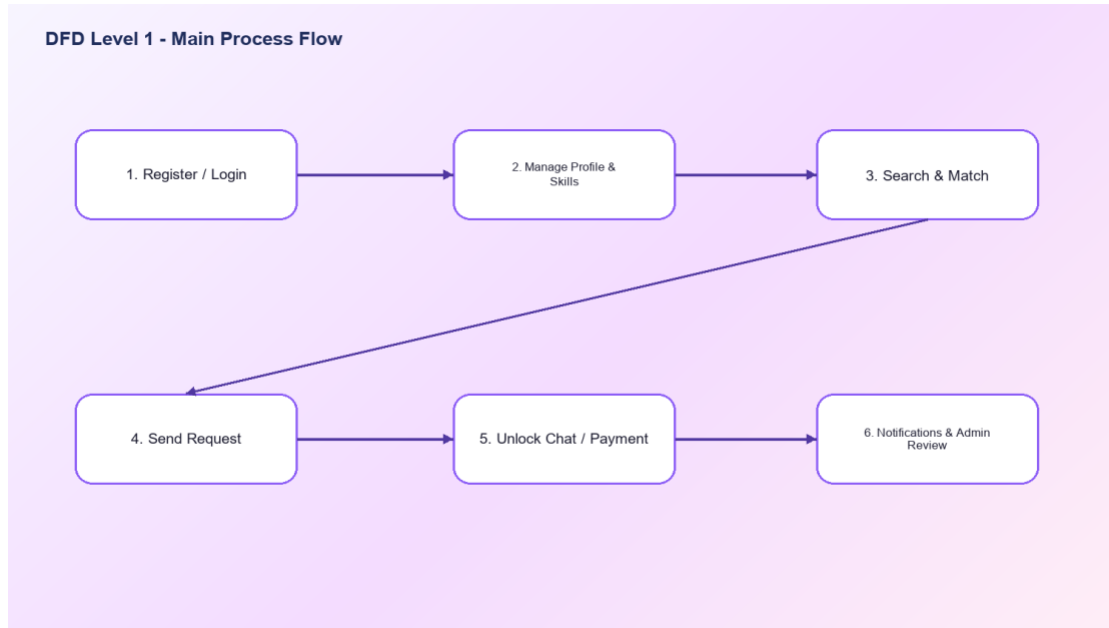


Figure: Database Relationship Diagram. This professionally recreated diagram uses clean box alignment and drawn connectors instead of text arrow symbols, so it remains printable and free from encoding corruption.

Database Relationship Diagram Explanation

Database Relationship Diagram Explanation in SkillFlow defines the boundary of the application by separating learner features, admin controls, database operations, payment supported access and report handling into clear functional areas.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

CHAPTER 6 - DATABASE DESIGN

Database Overview

Database Overview in SkillFlow discusses the problem that learners often know what they want to learn but do not have a reliable way to find someone who needs their own skill in return.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Table responsibility	Table responsibility is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Relationship	Relationship is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Validation	Validation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Security	Security is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Database Architecture

Database Architecture in SkillFlow shows how the development process was planned around requirement study, database design, route implementation, interface preparation, validation, testing and final deployment readiness.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Database Workflow

Database Workflow in SkillFlow explains how administrative automation supports the project by monitoring users, reports, requests, payments and platform activity from a controlled backend panel.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Database Connectivity

Database Connectivity in SkillFlow highlights the database driven nature of the system, where important actions are stored, retrieved and updated through MySQL tables instead of remaining temporary browser data.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Data Storage Process

Data Storage Process in SkillFlow connects digital learning requirements with practical software engineering by combining authentication, search, matching, messaging and analytics into a single student project.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

User Data Management

User Data Management in SkillFlow emphasizes user convenience through clean navigation, responsive pages, visible status messages, quick notifications and a dashboard that keeps major actions easy to reach.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Admin Data Management

Admin Data Management in SkillFlow studies the existing limitations of manual learning exchange and explains why a dedicated platform gives better tracking, accountability and communication control.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Skill Data Management

Skill Data Management in SkillFlow presents the proposed solution as a manageable web application where every major process is represented through clear routes, templates, forms and database relationships.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Request Data Management

Request Data Management in SkillFlow examines technical feasibility through the availability of Flask, MySQL, browser based frontend tools and a development environment that can be maintained on standard systems.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Payment Data Storage

Payment Data Storage in SkillFlow evaluates operational feasibility by considering whether normal users can register, add skills, send requests, unlock chat and understand notifications without technical training.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

OTP and Verification Storage

OTP and Verification Storage in SkillFlow discusses economic feasibility because the project can be built with open source technologies while still supporting premium style features such as plans and payment records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Inventory Data Handling

Inventory Data Handling in SkillFlow organizes requirement analysis around real users, admin responsibilities, security checks, database fields and the communication flow required for learning exchanges.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Session Management

Session Management in SkillFlow presents system design as the bridge between project idea and implementation, converting modules into architecture, diagrams, tables, pages and backend workflows.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Notifications Data Flow

Notifications Data Flow in SkillFlow describes the architecture through the browser, Flask application layer, route controllers, MySQL database and admin modules that cooperate during every transaction.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Reports and Analytics Data Flow

Reports and Analytics Data Flow in SkillFlow explains data movement by showing how inputs from forms become requests, matches, messages, notifications, reviews, reports and analytics records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Normalization and Integrity

Normalization and Integrity in SkillFlow represents entity relationships among users, skills, requests, matches, payments, messages and reports so that database dependency can be understood clearly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Actual Database Source: Tables were generated from database/skillflow_schema.sql and Python Flask CREATE TABLE statements. Request List/saved profile are not shown as persistent MySQL tables when not present in parsed project database files.

Figure 6.A: Python FlaskMyAdmin Database Overview - Table List Part 1

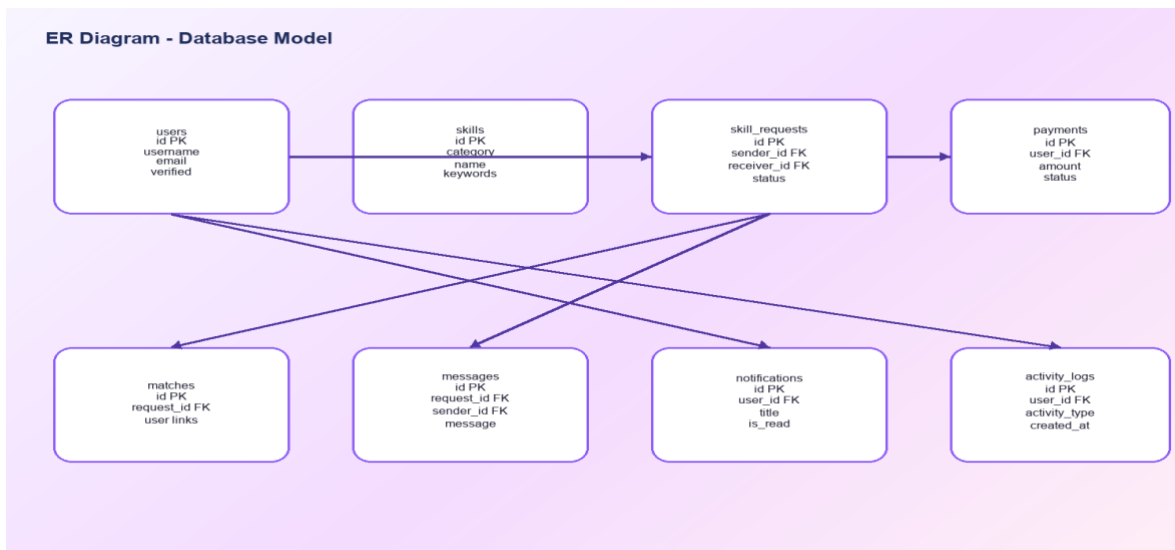


Figure 6.A: Python FlaskMyAdmin Database Overview - Table List Part 1. This actual Python FlaskMyAdmin screenshot is inserted as database evidence for the SkillFlow project and shows real table/database structure used during development.

Figure 6.B: Python FlaskMyAdmin Database Overview - Table List Part 2

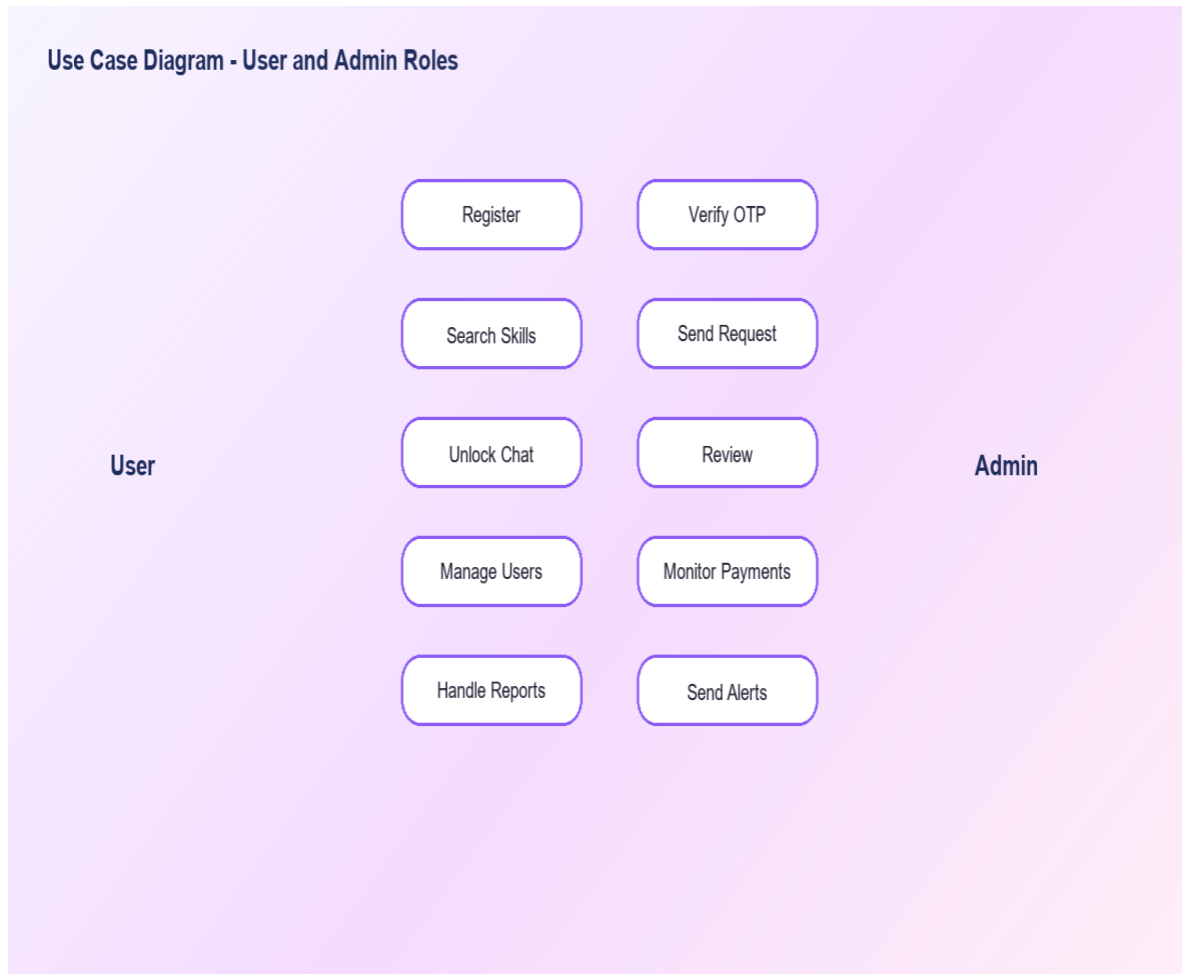


Figure 6.B: Python FlaskMyAdmin Database Overview - Table List Part 2. This actual Python FlaskMyAdmin screenshot is inserted as database evidence for the SkillFlow project and shows real table/database structure used during development.

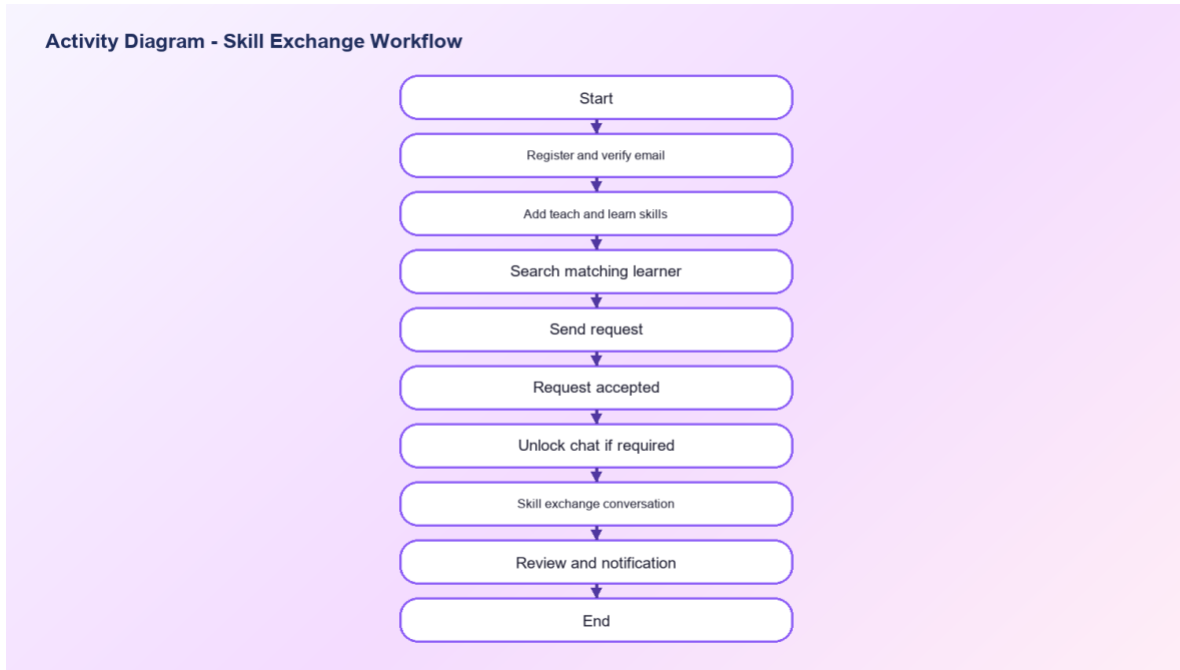
Figure 6.C: Python FlaskMyAdmin Database Overview - Table List Part 3

Figure 6.C: Python FlaskMyAdmin Database Overview - Table List Part 3. This actual Python FlaskMyAdmin screenshot is inserted as database evidence for the SkillFlow project and shows real table/database structure used during development.

From Table	Column	To Table	To Column	Basis
admin_login_activity	admin_id	admin_accounts	admin_id	inferred from column naming
reports	admin_id	admin_accounts	admin_id	inferred from column naming
user_notifications	skill_id	skills	id	inferred from column naming
user_notifications	notification_id	notifications	id	inferred from column naming
user_notifications	user_id	users	id	inferred from column naming

notifications	user_id	users	id	inferred from column naming
notifications	skill_id	skills	id	inferred from column naming
notifications	request_id	requests	request_id	inferred from column naming
messages	request_id	requests	request_id	inferred from column naming
messages	skill_id	skills	id	inferred from column naming
unfollow_reports	request_id	requests	request_id	inferred from column naming
unfollow_reports	user_id	users	id	inferred from column naming
requests	user_id	users	id	inferred from column naming
requests	skill_id	skills	id	inferred from column naming
requests	payment_id	payments	id	inferred from column naming
payments	request_id	requests	request_id	inferred from column naming
payments	user_id	users	id	inferred from column naming
user_skill_categories	skill_id	skills	id	inferred from column naming

user_skill_categories	category_id	skill_categories	id	inferred from column naming
user_skill_categories	subcategory_id	subskill_categories	id	inferred from column naming
user_reviews	user_id	users	id	inferred from column naming
user_reviews	skill_id	skills	id	inferred from column naming
skills	category_id	skill_categories	id	inferred from column naming
skills	subcategory_id	subskill_categories	id	inferred from column naming
reviews	user_id	users	id	inferred from column naming
reviews	skill_id	skills	id	inferred from column naming
reviews	request_id	requests	request_id	inferred from column naming
user_notifications	user_id	users	id	inferred from column naming
user_settings	user_id	users	id	inferred from column naming
subskill_categories	category_id	skill_categories	id	inferred from column naming

Table 6.1: admin_login_activity

Admin login audit trail.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
admin_id	INT	11	No	Inferred	NOT NULL	Primary identifier for this record.
session_id	VARCHAR	128	No	No	NOT NULL	Reference identifier connecting this record with another module.
ip_address	VARCHAR	60	No	No	DEFAULT NULL	Business field used by the admin login activity module.
user_agent	VARCHAR	255	No	No	DEFAULT NULL	Business field used by the admin login activity module.
login_time	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
logout_time	DATETIME		No	No	DEFAULT NULL	Timestamp for audit,

						sorting and reports.
is_active	TINYINT	1	No	No	DEFAULT 1	Business field used by the admin login activity module.

The admin_login_activity table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Figure 6.1: Admin Login Activity Table Structure

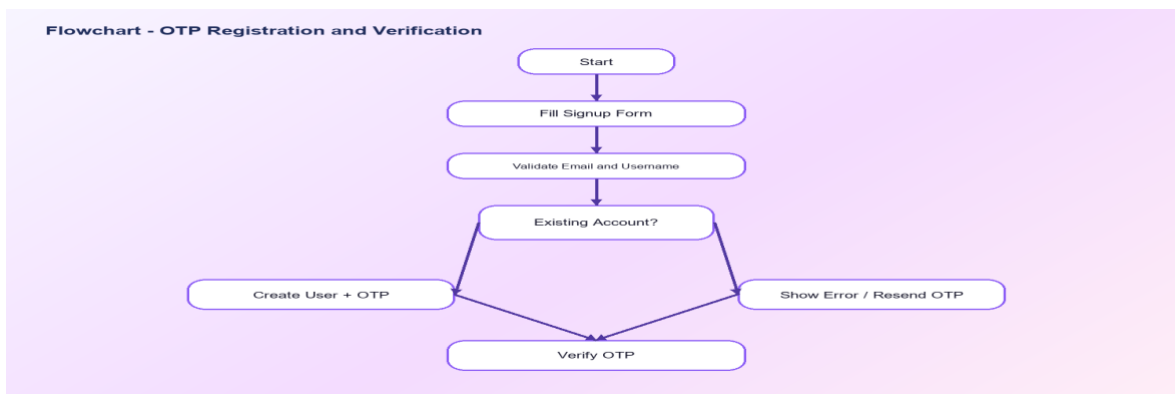


Figure 6.1: Admin Login Activity Table Structure. This actual Python FlaskMyAdmin screenshot is inserted as database evidence for the SkillFlow project and shows real table/database structure used during development.

Table 6.2: admin_notifications

Admin panel alerts and internal messages.

Field Name	Type	Size	PK	FK	Constraints	Description
------------	------	------	----	----	-------------	-------------

id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
title	VARCHAR	180	No	No	NOT NULL	Display label shown in interface.
message	TEXT		No	No	NOT NULL	Business field used by the admin notifications module.
type	VARCHAR	60	No	No	NOT NULL	Business field used by the admin notifications module.
source_key	VARCHAR	180	No	No	DEFAULT NULL	Business field used by the admin notifications module.
is_read	TINYINT	1	No	No	DEFAULT 0	Business field used by the admin notifications module.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
reference_id	VARCHAR	120	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.

reference_type	VARCHAR	60	No	No	DEFAULT NULL	Business field used by the admin notifications module.
is_deleted	TINYINT	1	No	No	DEFAULT 0	Business field used by the admin notifications module.
deleted_at	DATETIME		No	No	DEFAULT NULL	Business field used by the admin notifications module.

The admin_notifications table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.3: reports

Saved/derived report summaries.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
admin_id	INT	11	No	Inferred	DEFAULT NULL	Primary identifier for this record.
admin_name	VARCHAR	120	No	No	DEFAULT NULL	Display label shown in interface.

admin_email	VARCHAR	160	No	No	DEFAULT NULL	Email used for login, verification or communication .
action_type	VARCHAR	80	No	No	NOT NULL	Business field used by the admin reports module.
description	TEXT		No	No	NOT NULL	Business field used by the admin reports module.
item_type	VARCHAR	80	No	No	DEFAULT NULL	Business field used by the admin reports module.
item_id	VARCHAR	120	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.
item_name	VARCHAR	180	No	No	DEFAULT NULL	Display label shown in interface.
ip_address	VARCHAR	60	No	No	DEFAULT NULL	Business field used by the admin reports module.
user_agent	VARCHAR	255	No	No	DEFAULT NULL	Business field used by the admin reports module.

created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
------------	----------	--	----	----	--------------------------------	---

The reports table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.4: admin_settings

Admin panel configurable settings.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
setting_key	VARCHAR	120	No	No	NOT NULL	Business field used by the admin settings module.
setting_value	TEXT		No	No	DEFAULT NULL	Business field used by the admin settings module.
updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.

The admin_settings table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.5: admin_accounts

Admin identities, roles and secure dashboard login data.

Field Name	Type	Size	PK	FK	Constraints	Description
admin_id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
name	VARCHAR	100	No	No	NOT NULL	Display label shown in interface.
email	VARCHAR	150	No	No	NOT NULL	Email used for login, verification or communication.
username	VARCHAR	80	No	No	NOT NULL	Display label shown in interface.
password	VARCHAR	255	No	No	NOT NULL	Password field; project security requires hashed storage.
created_at	TIMESTAMP		No	No	NOT NULL DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
phone	VARCHAR	30	No	No	DEFAULT NULL	Contact number for user/admin communication.
profile_image	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.
last_login	DATETIME		No	No	DEFAULT NULL	Business field used by the

						admin_accounts module.
reset_otp	VARCHAR	10	No	No	DEFAULT NULL	Business field used by the admin_accounts module.
reset_otp_expiry	DATETIME		No	No	DEFAULT NULL	Business field used by the admin_accounts module.

The admin_accounts table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.6: skill_categories

Skill category records used for skill discovery and dashboard navigation.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
category_name	VARCHAR	120	No	No	NOT NULL	Display label shown in interface.
slug	VARCHAR	160	No	No	NOT NULL	Business field used by the skill_categories module.
parent_id	INT	11	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.

description	TEXT		No	No	DEFAULT NULL	Business field used by the skill_categories module.
category_image	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.
is_featured	TINYINT	1	No	No	DEFAULT 0	Business field used by the skill_categories module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin_accounts.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

The skill_categories table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.7: user_profiles

Public user_profiles form enquiries.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.

name	VARCHAR	120	No	No	NOT NULL	Display label shown in interface.
email	VARCHAR	160	No	No	NOT NULL	Email used for login, verification or communication.
message	TEXT		No	No	NOT NULL	Business field used by the user_profiles module.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

The user_profiles table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.8: skill_categories

Skill category and dashboard discovery content.

Field Name	Type	Size	PK	F K	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
image_path	VARCHAR	255	No	No	NOT NULL	Uploaded media path or filename.
title	VARCHAR	160	No	No	DEFAULT 'SkillFlow Luxury Living'	Display label shown in interface.
subtitle	VARCHAR	255	No	No	DEFAULT 'Elegant SkillFlows'	Display label shown in interface.

button_text	VARCHAR	80	No	No	DEFAULT 'Explore Collection'	Business field used by the hero slides module.
button_link	VARCHAR	255	No	No	DEFAULT '#skill_categories'	Business field used by the hero slides module.
image_fit	VARCHAR	20	No	No	DEFAULT 'cover'	Uploaded media path or filename.
image_position	VARCHAR	30	No	No	DEFAULT 'center'	Uploaded media path or filename.
sort_request	INT	11	No	No	DEFAULT 0	Business field used by the hero slides module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin_accounts.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.

The skill_categories table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.9: user_notifications

User notification records for platform updates and request activity.

Field Name	Type	Size	P K	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
offer_title	VARCHAR	180	No	No	NOT NULL	Display label shown in interface.
discount_text	VARCHAR	120	No	No	DEFAULT "	Business field used by the SkillFlow discount offers module.
link_type	VARCHAR	20	No	No	DEFAULT 'category'	Business field used by the SkillFlow discount offers module.
skill_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
category_slug	VARCHAR	160	No	No	DEFAULT "	Business field used by the SkillFlow discount offers module.
start_date	DATE		No	No	DEFAULT NULL	Timestamp for audit, sorting and reports.

end_date	DATE		No	No	DEFAULT NULL	Timestamp for audit, sorting and reports.
image_path	VARCHAR	255	No	No	DEFAULT "	Uploaded media path or filename.
sort_request	INT	11	No	No	DEFAULT 0	Business field used by the SkillFlow discount offers module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin_accounts.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.

The user_notifications table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.10: user_activity

User activity and engagement records for dashboard analytics.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.

title	VARCHAR	160	No	No	NOT NULL	Display label shown in interface.
subtitle	VARCHAR	255	No	No	DEFAULT "	Display label shown in interface.
button_text	VARCHAR	80	No	No	DEFAULT "	Business field used by the SkillFlow feature boxes module.
button_link	VARCHAR	255	No	No	DEFAULT "	Business field used by the SkillFlow feature boxes module.
image_path	VARCHAR	255	No	No	DEFAULT "	Uploaded media path or filename.
image_fit	VARCHAR	20	No	No	DEFAULT 'cover'	Uploaded media path or filename.
image_position	VARCHAR	30	No	No	DEFAULT 'center'	Uploaded media path or filename.
sort_request	INT	11	No	No	DEFAULT 0	Business field used by the SkillFlow feature boxes module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin accounts.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.
------------	----------	--	----	----	--	---

The user_activity table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.11: admin_settings

Admin configurable settings used by the SkillFlow platform.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL DEFAULT 1	Primary identifier for this record.
offer_text	VARCHAR	500	No	No	NOT NULL DEFAULT "	Business field used by the SkillFlow offer strip module.
enabled	TINYINT	1	No	No	DEFAULT 1	Business field used by the SkillFlow offer strip module.
marquee_speed	INT	11	No	No	DEFAULT 28	Business field used by the SkillFlow offer strip module.
updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.

The admin_settings table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.12: matches

Learning-based Dashboard learning cards.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
title	VARCHAR	160	No	No	NOT NULL	Display label shown in interface.
image_path	VARCHAR	255	No	No	DEFAULT "	Uploaded media path or filename.
link_url	VARCHAR	255	No	No	DEFAULT "	Business field used by the SkillFlow learning cards module.
sort_request	INT	11	No	No	DEFAULT 0	Business field used by the SkillFlow learning cards module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin accounts.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.

The matches table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.13: user_reviews

Dashboard video banner metadata.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
video_path	VARCHAR	255	No	No	DEFAULT "	Business field used by the SkillFlow video banner module.
heading	VARCHAR	160	No	No	DEFAULT 'SkillFlow Luxury Living'	Business field used by the SkillFlow video banner module.
subtitle	VARCHAR	255	No	No	DEFAULT 'Premium skill crafted for modern	Display label shown in interface.
button_text	VARCHAR	80	No	No	DEFAULT 'Explore Collection'	Business field used by the SkillFlow video banner module.
button_link	VARCHAR	255	No	No	DEFAULT 'skill-list.py'	Business field used by the SkillFlow video banner module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin accounts.

updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.
sort_request	INT	11	No	No	DEFAULT 0	Business field used by the SkillFlow video banner module.

The user_reviews table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.14: user_favorites

Value proposition points.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
icon_class	VARCHAR	120	No	No	DEFAULT 'fa-solid fa-gem'	Business field used by the SkillFlow why points module.
title	VARCHAR	160	No	No	NOT NULL	Display label shown in interface.
description	TEXT		No	No	DEFAULT NULL	Business field used by the SkillFlow why points module.
sort_request	INT	11	No	No	DEFAULT 0	Business field used by the SkillFlow why points module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to

						users/admin_accounts.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.

The user_favorites table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.15: user_notifications

Read/unread state for notifications.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
notification_id	INT	11	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
user_id	INT	11	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.

read_at	DATETIME		No	No	DEFAULT current_timestamp()	Business field used by the notification reads module.
---------	----------	--	----	----	--------------------------------	---

The user_notifications table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.16: notifications

User/admin notifications for requests, account actions and updates.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
user_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
title	VARCHAR	180	No	No	NOT NULL	Display label shown in interface.
message	TEXT		No	No	NOT NULL	Business field used by the notifications module.
type	VARCHAR	40	No	No	DEFAULT 'system'	Business field used by the notifications module.
is_read	TINYINT	1	No	No	DEFAULT 0	Business field used by the notifications module.

created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin_accounts.
is_visible	TINYINT	1	No	No	DEFAULT 1	Business field used by the notifications module.
skill_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
request_id	VARCHAR	60	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
link_url	VARCHAR	255	No	No	DEFAULT NULL	Business field used by the notifications module.
target_type	VARCHAR	50	No	No	DEFAULT NULL	Business field used by the notifications module.
target_id	VARCHAR	100	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.

The notifications table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.17: messages

Line items belonging to an request with skill quantity and price snapshot.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
request_id	VARCHAR	40	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
skill_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
skill_name	VARCHAR	180	No	No	DEFAULT NULL	Display label shown in interface.
skill_image	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.
quantity	INT	11	No	No	DEFAULT 1	Quantity used for stock/request calculations.
price	DECIMAL		No	No		Monetary value used in skill discovery, request or payment calculation.

subtotal	DECIMAL		No	No		Monetary value used in skill discovery, request or payment calculation.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

The messages table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.18: unfollow_reports

Return requests and reasons.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
request_id	VARCHAR	40	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
user_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
reason	VARCHAR	160	No	No	DEFAULT NULL	Reason recorded for

						action transparency.
comment	TEXT		No	No	DEFAULT NULL	Business field used by the request returns module.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

The unfollow_reports table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.19: requests

Request headers with user, amount, delivery and fulfilment status.

Field Name	Type	Size	P K	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
request_id	VARCHAR	40	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.
request_code	VARCHAR	40	No	No	DEFAULT NULL	Business field used by the requests module.
user_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.

skill_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
user_name	VARCHAR	120	No	No	DEFAULT NULL	Display label shown in interface.
user_email	VARCHAR	160	No	No	DEFAULT NULL	Email used for login, verification or communication.
user_phone	VARCHAR	30	No	No	DEFAULT NULL	Contact number for user/admin communication.
user_user_profiles	VARCHAR	30	No	No	DEFAULT NULL	Business field used by the requests module.
delivery_address	TEXT		No	No	DEFAULT NULL	Business field used by the requests module.
payment_status	VARCHAR	50	No	No	DEFAULT 'Pending'	Workflow state shown to users/admin_accounts.
request_status	VARCHAR	50	No	No	DEFAULT 'Pending'	Workflow state shown to users/admin_accounts.
request_date	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

confirmed_date	DATETIME		No	No	DEFAULT NULL	Timestamp for audit, sorting and reports.
shipping_date	DATETIME		No	No	DEFAULT NULL	Timestamp for audit, sorting and reports.
delivery_date	DATETIME		No	No	DEFAULT NULL	Timestamp for audit, sorting and reports.
return_date	DATETIME		No	No	DEFAULT NULL	Timestamp for audit, sorting and reports.
refund_date	DATETIME		No	No	DEFAULT NULL	Timestamp for audit, sorting and reports.
skill_name	VARCHAR	180	No	No	DEFAULT NULL	Display label shown in interface.
quantity	INT	11	No	No	DEFAULT 1	Quantity used for stock/request calculations.
payment_method	VARCHAR	80	No	No	DEFAULT NULL	Business field used by the requests module.
total	DECIMAL		No	No		Monetary value used in skill discovery, request or payment calculation.
total_amount	DECIMAL		No	No		Monetary value used in skill discovery, request

						or payment calculation.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
payment_id	VARCHAR	255	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
transaction_id	VARCHAR	255	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.
paid_at	DATETIME		No	No	DEFAULT NULL	Business field used by the requests module.

The requests table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Figure 6.19: Requests Table Structure

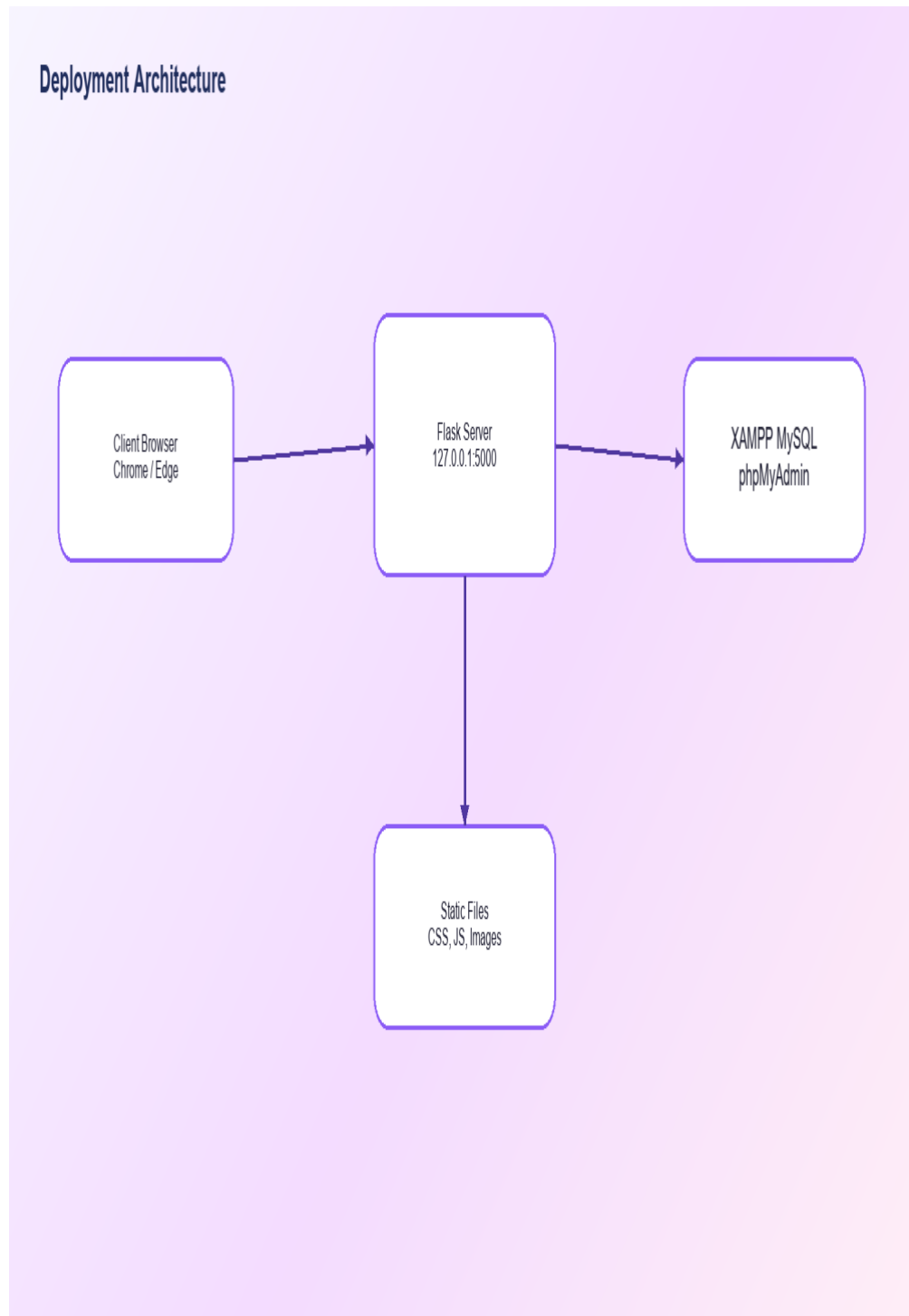


Figure 6.19: Requests Table Structure. This actual Python FlaskMyAdmin screenshot is inserted as database evidence for the SkillFlow project and shows real table/database structure used during development.

Table 6.20: payments

Payment attempt, transaction status and gateway verification data.

Field Name	Type	Size	P K	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
request_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
user_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
payment_gateway	VARCHAR	50	No	No	DEFAULT 'PhonePe'	Business field used by the payments module.
gateway_request_id	VARCHAR	255	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.
gateway_payment_id	VARCHAR	255	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.

transaction_id	VARCHAR	255	No	No	DEFAULT NULL	Reference identifier connecting this record with another module.
amount	DECIMAL		No	No		Monetary value used in skill discovery, request or payment calculation.
status	VARCHAR	50	No	No	DEFAULT 'pending'	Workflow state shown to users/admin accounts.
response_json	TEXT		No	No	DEFAULT NULL	Business field used by the payments module.
created_at	TIMESTAMP		No	No	NOT NULL DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

The payments table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.21: user_skill_categories

Multiple skill/category images stored outside database as paths.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.

skill_id	INT	11	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
category_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
subcategory_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
image_path	VARCHAR	255	No	No	NOT NULL	Uploaded media path or filename.
is_featured	TINYINT	1	No	No	DEFAULT 0	Business field used by the skill images module.
sort_request	INT	11	No	No	DEFAULT 0	Business field used by the skill images module.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit,

						sorting and reports.
is_main	TINYINT	1	No	No	DEFAULT 0	Business field used by the skill images module.

The user_skill_categories table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.22: user_reviews

Skill-specific rating records.

Field Name	Type	Size	P K	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
user_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
skill_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
user_name	VARCHAR	120	No	No	DEFAULT NULL	Display label shown in interface.
skill_name	VARCHAR	180	No	No	DEFAULT NULL	Display label shown in interface.

rating	INT	11	No	No	DEFAULT 5	Business field used by the skill reviews module.
review_message	TEXT		No	No	DEFAULT NULL	Business field used by the skill reviews module.
status	VARCHAR	30	No	No	DEFAULT 'Pending'	Workflow state shown to users/admin_accounts.
is_reported	TINYINT	1	No	No	DEFAULT 0	Business field used by the skill reviews module.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

The user_reviews table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.23: skills

Skill skill discovery records, pricing, stock, category mapping and images.

Field Name	Type	Size	P K	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
name	VARCHAR	180	No	No	NOT NULL	Display label shown in interface.
price	DECIMAL		No	No		Monetary value used in skill

						discovery, request or payment calculation.
category	VARCHAR	120	No	No	NOT NULL	Business field used by the skills module.
category_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
subcategory_id	INT	11	No	Inferred	DEFAULT NULL	Reference identifier connecting this record with another module.
image	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.
description	TEXT		No	No	DEFAULT NULL	Business field used by the skills module.
slug	VARCHAR	160	No	No	DEFAULT NULL	Business field used by the skills module.
brand	VARCHAR	120	No	No	DEFAULT NULL	Business field used by the skills module.
sku	VARCHAR	80	No	No	DEFAULT NULL	Business field used by the skills module.
original_price	DECIMAL		No	No		Monetary value used in skill discovery, request

						or payment calculation.
discount_price	DECIMAL		No	No		Monetary value used in skill discovery, request or payment calculation.
stock_quantity	INT	11	No	No	DEFAULT 0	Quantity used for stock/request calculations.
short_description	TEXT		No	No	DEFAULT NULL	Business field used by the skills module.
full_description	TEXT		No	No	DEFAULT NULL	Business field used by the skills module.
specifications	TEXT		No	No	DEFAULT NULL	Business field used by the skills module.
material	VARCHAR	120	No	No	DEFAULT NULL	Business field used by the skills module.
color	VARCHAR	80	No	No	DEFAULT NULL	Business field used by the skills module.
dimensions	VARCHAR	120	No	No	DEFAULT NULL	Business field used by the skills module.
weight	VARCHAR	80	No	No	DEFAULT NULL	Business field used by the skills module.

seating_capacity	VARCHAR	80	No	No	DEFAULT NULL	Business field used by the skills module.
learning_type	VARCHAR	120	No	No	DEFAULT NULL	Business field used by the skills module.
assembly_required	VARCHAR	20	No	No	DEFAULT 'No'	Business field used by the skills module.
featured	TINYINT	1	No	No	DEFAULT 0	Business field used by the skills module.
trending	TINYINT	1	No	No	DEFAULT 0	Business field used by the skills module.
in_stock	TINYINT	1	No	No	DEFAULT 1	Quantity used for stock/request calculations.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin_accounts.
gallery_images	TEXT		No	No	DEFAULT NULL	Uploaded media path or filename.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.
image_1	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.
image_2	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.

image_3	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.
image_4	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.

The skills table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.24: reviews

Review and rating data given by users.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
user_id	INT	11	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
skill_id	INT	11	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
request_id	VARCHAR	60	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
rating	INT	11	No	No	NOT NULL	Business field used by the reviews module.

review_text	TEXT		No	No	DEFAULT NULL	Business field used by the reviews module.
status	VARCHAR	20	No	No	DEFAULT 'pending'	Workflow state shown to users/admin accounts.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

The reviews table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.25: user_notifications

Saved user delivery addresses for chat unlock.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
user_id	INT	11	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
full_name	VARCHAR	120	No	No	NOT NULL	Display label shown in interface.
phone	VARCHAR	30	No	No	NOT NULL	Contact number for user/admin

						communication .
house_no	VARCHAR	120	No	No	NOT NULL	Business field used by the user addresses module.
street_address	VARCHAR	180	No	No	NOT NULL	Business field used by the user addresses module.
city	VARCHAR	80	No	No	NOT NULL	Business field used by the user addresses module.
state	VARCHAR	80	No	No	NOT NULL	Business field used by the user addresses module.
pincode	VARCHAR	20	No	No	NOT NULL	Business field used by the user addresses module.
landmark	VARCHAR	180	No	No	DEFAULT NULL	Business field used by the user addresses module.
created_at	DATETIME		No	No	DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

The user_notifications table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.26: user_settings

User preferences and account settings.

Field Name	Type	Size	PK	FK	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
user_id	INT	11	No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
notifications	TINYINT	1	No	No	DEFAULT 1	Business field used by the user settings module.
privacy_options	TINYINT	1	No	No	DEFAULT 0	Business field used by the user settings module.
account_preferences	TINYINT	1	No	No	DEFAULT 1	Business field used by the user settings module.
updated_at	DATETIME		No	No	DEFAULT current_timestamp() ON UPDATE current	Timestamp for audit, sorting and reports.

The user_settings table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.27: users

Registered user profile, verification, login and account-control data.

Field Name	Type	Size	P K	F K	Constraints	Description
id	INT	11	Yes	No	NOT NULL AUTO_INCREMENT	Primary identifier for this record.
full_name	VARCHAR	120	No	No	NOT NULL	Display label shown in interface.
username	VARCHAR	80	No	No	NOT NULL	Display label shown in interface.
email	VARCHAR	160	No	No	NOT NULL	Email used for login, verification or communication.
phone	VARCHAR	30	No	No	DEFAULT NULL	Contact number for user/admin communication.
gender	VARCHAR	30	No	No	DEFAULT NULL	Business field used by the users module.
profile_image	VARCHAR	255	No	No	DEFAULT NULL	Uploaded media path or filename.
password	VARCHAR	255	No	No	NOT NULL	Password field; project security requires hashed storage.

address	TEXT		No	No	DEFAULT NULL	Business field used by the users module.
city	VARCHAR	80	No	No	DEFAULT NULL	Business field used by the users module.
state	VARCHAR	80	No	No	DEFAULT NULL	Business field used by the users module.
pincode	VARCHAR	20	No	No	DEFAULT NULL	Business field used by the users module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin_accounts.
is_blocked	TINYINT	1	No	No	DEFAULT 0	Business field used by the users module.
is_deleted	TINYINT	1	No	No	DEFAULT 0	Business field used by the users module.
deleted_at	DATETIME		No	No	DEFAULT NULL	Business field used by the users module.
last_login	DATETIME		No	No	DEFAULT NULL	Business field used by the users module.
created_at	TIMESTAMP		No	No	NOT NULL DEFAULT current_timestamp()	Timestamp for audit, sorting and reports.

profile_image_position_x	DECIMAL		No	No		Uploaded media path or filename.
profile_image_position_y	DECIMAL		No	No		Uploaded media path or filename.
profile_image_zoom	DECIMAL		No	No		Uploaded media path or filename.
remember_token_hash	VARCHAR	255	No	No	DEFAULT NULL	Business field used by the users module.
remember_token_expires	DATETIME		No	No	DEFAULT NULL	Business field used by the users module.
reset_otp	VARCHAR	10	No	No	DEFAULT NULL	Business field used by the users module.
reset_otp_expiry	DATETIME		No	No	DEFAULT NULL	Business field used by the users module.

The users table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Figure 6.27: Users Table Structure

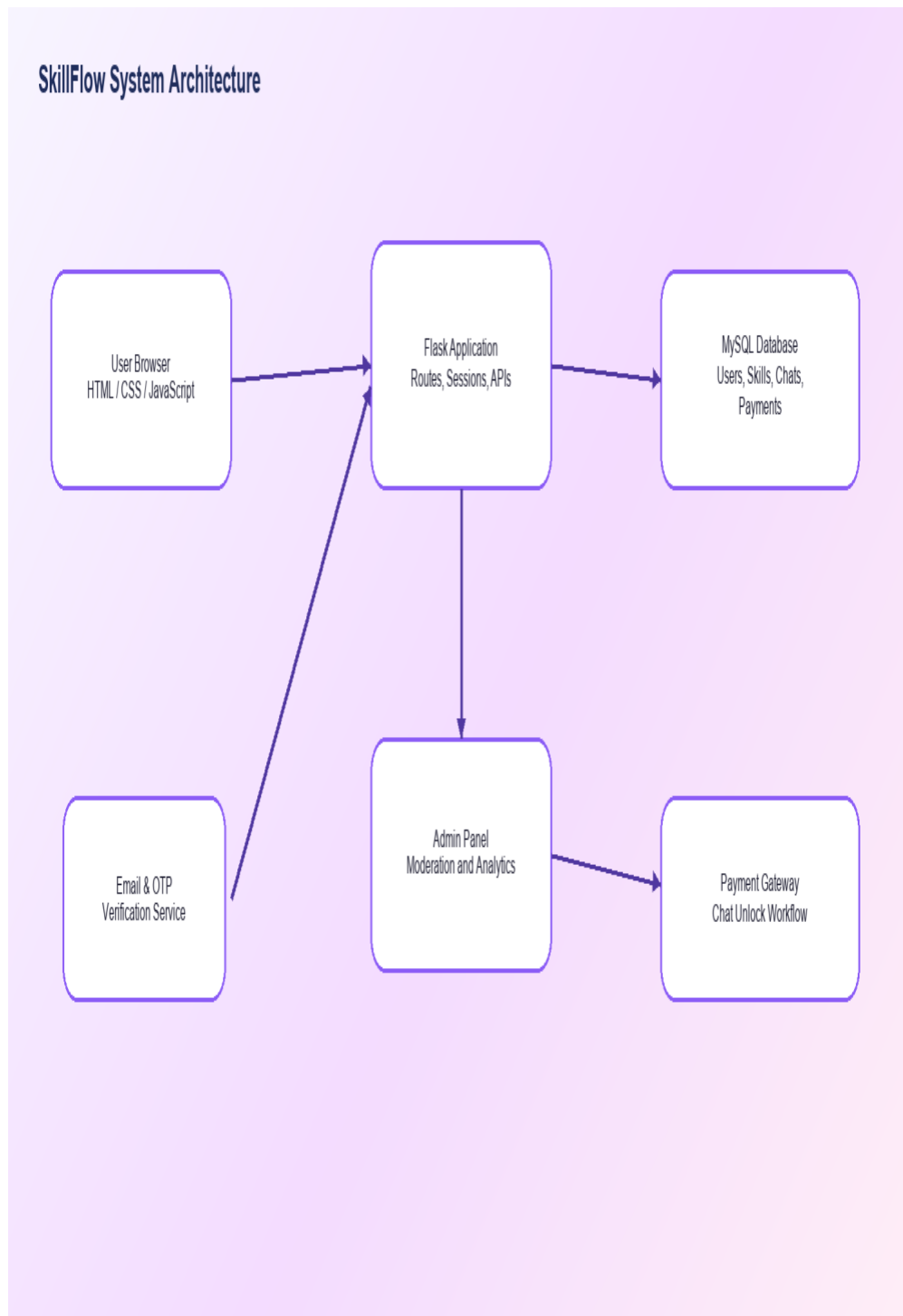


Figure 6.27: Users Table Structure. This actual Python FlaskMyAdmin screenshot is inserted as database evidence for the SkillFlow project and shows real table/database structure used during development.

Table 6.28: subskill_categories

Child skill groups used for skill classification.

Field Name	Type	Size	P K	FK	Constraints	Description
id	INT		No	No	AUTO_INCREMENT PRIMARY KEY	Primary identifier for this record.
old_category_id	INT		No	No	NULL UNIQUE	Reference identifier connecting this record with another module.
category_id	INT		No	Inferred	NOT NULL	Reference identifier connecting this record with another module.
subcategory_name	VARCHAR	120	No	No	NOT NULL	Display label shown in interface.
slug	VARCHAR	160	No	No	NOT NULL UNIQUE	Business field used by the subskill_categories module.
description	TEXT		No	No	NULL	Business field used by the subskill_categories module.
image	VARCHAR	255	No	No	NULL	Uploaded media path or filename.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to

						users/admin_accounts.
created_at	DATETIME		No	No	DEFAULT CURRENT_TIMESTAMP	Timestamp for audit, sorting and reports.

The subskill_categories table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.29: reports

Runtime-created editable Dashboard sections.

Field Name	Type	Size	P K	F K	Constraints	Description
section_key	VARCHAR	80	No	No	PRIMARY KEY	Business field used by the SkillFlow content sections module.
title	VARCHAR	160	No	No	NOT NULL	Display label shown in interface.
subtitle	VARCHAR	255	No	No	DEFAULT "	Display label shown in interface.
button_text	VARCHAR	80	No	No	DEFAULT "	Business field used by the SkillFlow content sections module.
button_link	VARCHAR	255	No	No	DEFAULT "	Business field used by the SkillFlow content sections module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin_accounts.

updated_at	DATETIME		No	No	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Timestamp for audit, sorting and reports.
------------	----------	--	----	----	---	---

The reports table participates in the database layer by storing normalized records instead of hard-coded page data. Where formal foreign keys are absent, relationships are documented as inferred from field names and should be enforced in future migrations.

Table 6.30: admin_actions

Runtime-created items under Dashboard sections.

Field Name	Type	Size	P K	F K	Constraints	Description
id	INT		No	No	AUTO_INCREMENT PRIMARY KEY	Primary identifier for this record.
section_key	VARCHAR	80	No	No	NOT NULL	Business field used by the SkillFlow content items module.
title	VARCHAR	180	No	No	NOT NULL	Display label shown in interface.
subtitle	VARCHAR	255	No	No	DEFAULT "	Display label shown in interface.
price_text	VARCHAR	80	No	No	DEFAULT "	Monetary value used in skill discovery, request or payment calculation.
button_text	VARCHAR	80	No	No	DEFAULT "	Business field used by the SkillFlow content items module.

button_link	VARCHAR	255	No	No	DEFAULT "	Business field used by the SkillFlow content items module.
image_path	VARCHAR	255	No	No	DEFAULT "	Uploaded media path or filename.
sort_request	INT		No	No	DEFAULT 0	Business field used by the SkillFlow content items module.
status	VARCHAR	20	No	No	DEFAULT 'active'	Workflow state shown to users/admin_accounts.
created_at	DATETIME		No	No	DEFAULT CURRENT_TIMESTAMP	Timestamp for audit, sorting and reports.
updated_at	DATETIME		No	No	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_T	Timestamp for audit, sorting and reports.

CHAPTER 7 - TECHNOLOGIES USED

Python Flask

Python Flask in SkillFlow documents user behaviour through UML style views that make login, profile editing, request sending, chat access and admin review easier to explain during viva.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Frontend action	Frontend action is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Backend handling	Backend handling is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Database use	Database use is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Security	Security is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

MySQL

MySQL in SkillFlow describes database design with attention to field purpose, relationship meaning, integrity and the way MySQL supports persistent project records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

HTML

HTML in SkillFlow explains each module by connecting the page seen by the user with the route, validation checks and table operations performed in the backend.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

CSS

CSS in SkillFlow presents authentication as the first controlled entry point where user identity, session values and validation rules protect the rest of the application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

JavaScript

JavaScript in SkillFlow explains OTP verification as an additional confirmation step that improves trust before sensitive account actions are accepted by the system.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Bootstrap

Bootstrap in SkillFlow studies skill matching as the main business logic where offered skills, required skills and user interests are compared to suggest meaningful exchange opportunities.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

local Flask/MySQL environment

local Flask/MySQL environment in SkillFlow explains the chat system as a controlled communication area that becomes useful after the system has enough information to connect two users responsibly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Python FlaskMyAdmin

Python FlaskMyAdmin in SkillFlow discusses payment integration through plan selection, transaction recording and administrative visibility rather than treating payment as a separate isolated feature.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Payment Gateway Integration

Payment Gateway Integration in SkillFlow describes the dashboard as the user's working area where personal details, skills, requests, matches, notifications and recent activity are brought together.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

OTP System

OTP System in SkillFlow presents the admin panel as the management side of the software where platform activity can be supervised through users, reports, plans, analytics and notifications.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Admin Panel

Admin Panel in SkillFlow explains validation by showing how forms, route checks, database constraints and user feedback reduce incorrect entries and incomplete submissions.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Responsive Design

Responsive Design in SkillFlow discusses security from the viewpoint of sessions, password handling, OTP checks, role separation, controlled routes and careful processing of user supplied values.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Sessions and Cookies

Sessions and Cookies in SkillFlow explains testing through realistic operations such as registration, login, skill posting, request creation, chat unlock, payment recording and admin review.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

File Upload Handling

File Upload Handling in SkillFlow presents results by connecting completed features with observable outputs on pages, database records and admin dashboard summaries.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

CHAPTER 8 - MODULE DESCRIPTION

User Authentication Module

User Authentication Module in SkillFlow describes advantages through practical benefits such as organized exchange, reduced manual search, better accountability and improved learning collaboration.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Frontend action	Frontend action is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Backend handling	Backend handling is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Database use	Database use is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Security	Security is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Home Page Module

Home Page Module in SkillFlow discusses limitations honestly, including the dependency on correct user data, internet availability and future scope for stronger recommendation intelligence.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Skill Browsing Module

Skill Browsing Module in SkillFlow presents future scope by identifying improvements such as AI matching, mobile application support, advanced analytics, richer communication and scalable deployment.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Category and Subcategory Module

Category and Subcategory Module in SkillFlow concludes the project by summarizing how the planned learning exchange system was converted into a working Flask and MySQL based web application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Skill Detail Module

Skill Detail Module in SkillFlow introduces how the platform converts informal peer learning into an organized digital workflow where users can publish skills, discover compatible learners and begin structured exchanges through a Flask powered interface.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Frontend action	Frontend action is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Backend handling	Backend handling is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Database use	Database use is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Security	Security is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Search and Filter Module

Search and Filter Module in SkillFlow explains the learning exchange idea from the user's point of view, showing how a person moves from registration to skill discovery, request creation, matching and controlled communication.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Saved Profile Module

Saved Profile Module in SkillFlow focuses on the practical need for a web based environment where students and professionals can trade knowledge without depending on offline contacts or scattered social media messages.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Request List Module

Request List Module in SkillFlow describes the project's core purpose: to bring skill listing, matching, request handling, chat access, notifications and administrative supervision into one connected application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Chat Unlock Module

Chat Unlock Module in SkillFlow presents the academic and technical goals of the project, especially the use of Python Flask, HTML, CSS, JavaScript and MySQL to solve a real collaboration problem.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Payment Module

Payment Module in SkillFlow defines the boundary of the application by separating learner features, admin controls, database operations, payment supported access and report handling into clear functional areas.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Request Tracking Module

Request Tracking Module in SkillFlow discusses the problem that learners often know what they want to learn but do not have a reliable way to find someone who needs their own skill in return.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Profile Module

Profile Module in SkillFlow shows how the development process was planned around requirement study, database design, route implementation, interface preparation, validation, testing and final deployment readiness.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Notifications Module

Notifications Module in SkillFlow explains how administrative automation supports the project by monitoring users, reports, requests, payments and platform activity from a controlled backend panel.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Admin Dashboard Module

Admin Dashboard Module in SkillFlow highlights the database driven nature of the system, where important actions are stored, retrieved and updated through MySQL tables instead of remaining temporary browser data.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Manage Skills Module

Manage Skills Module in SkillFlow connects digital learning requirements with practical software engineering by combining authentication, search, matching, messaging and analytics into a single student project.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Manage Categories Module

Manage Categories Module in SkillFlow emphasizes user convenience through clean navigation, responsive pages, visible status messages, quick notifications and a dashboard that keeps major actions easy to reach.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Manage Requests Module

Manage Requests Module in SkillFlow studies the existing limitations of manual learning exchange and explains why a dedicated platform gives better tracking, accountability and communication control.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Manage Users Module

Manage Users Module in SkillFlow presents the proposed solution as a manageable web application where every major process is represented through clear routes, templates, forms and database relationships.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Payment Management Module

Payment Management Module in SkillFlow examines technical feasibility through the availability of Flask, MySQL, browser based frontend tools and a development environment that can be maintained on standard systems.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Reports and Analytics Module

Reports and Analytics Module in SkillFlow evaluates operational feasibility by considering whether normal users can register, add skills, send requests, unlock chat and understand notifications without technical training.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Inventory Management Module

Inventory Management Module in SkillFlow discusses economic feasibility because the project can be built with open source technologies while still supporting premium style features such as plans and payment records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Home Page Editor Module

Home Page Editor Module in SkillFlow organizes requirement analysis around real users, admin responsibilities, security checks, database fields and the communication flow required for learning exchanges.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Banner and Slider Management Module

Banner and Slider Management Module in SkillFlow presents system design as the bridge between project idea and implementation, converting modules into architecture, diagrams, tables, pages and backend workflows.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Activity Logs Module

Activity Logs Module in SkillFlow describes the architecture through the browser, Flask application layer, route controllers, MySQL database and admin modules that cooperate during every transaction.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Reviews Management Module

Reviews Management Module in SkillFlow explains data movement by showing how inputs from forms become requests, matches, messages, notifications, reviews, reports and analytics records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

CHAPTER 9 - CODING

This chapter presents selected Python Flask code used in the SkillFlow project. The snippets are taken from the project application file and are intentionally limited to important modules instead of copying the complete source code. Each section explains purpose, input, output, database interaction and security handling.

9.1 Database Connection

Module Name	Database Connection
File Name	app.py
Purpose	Creates a reusable MySQL connection for Flask routes.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	Database credentials are read by the connection helper and routes use cursor.execute for safe interaction.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

545 def get_db_connection():
546     try:
547         connection = pymysql.connect(
548             host=os.getenv('DB_HOST', 'localhost'),
549             user=os.getenv('DB_USER', 'root'),
550             password=os.getenv('DB_PASSWORD', ''),

```

```

551         database=os.getenv('DB_NAME', 'skillflow_db'),
552         cursorclass=pymysql.cursors.DictCursor
553     )
554
555
556     return connection
557 except Exception as e:
558     print(f"Error connecting to database: {e}")
559     return None
560
561
562 def transform_timestamp(value):
563     if isinstance(value, datetime):
564         return value.strftime('%Y-%m-%dT%H:%M:%S')
565     return value
566
567
568 USERNAME_PATTERN = re.compile(r'^[A-Za-z0-9_.]{3,25}$')
569 EMAIL_PATTERN = re.compile(r'^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$', re.IGNORECASE)
570 DISPOSABLE_EMAIL_DOMAINS = {
571     '10minutemail.com', '10minutemail.net', '10minutemail.org',
572     '20minutemail.com',
573     'guerrillamail.com', 'guerrillamail.net', 'guerrillamail.org',
574     'guerrillamail.de',
575     'yopmail.com', 'yopmail.fr', 'yopmail.net',
576     'mailinator.com', 'mailinator.net', 'mailinator.org',

```

Explanation: The database connection module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.2 User Registration

Module Name	User Registration
File Name	app.py

Purpose	Registers a new user, validates email, password and duplicate account state.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The route normalizes email, checks verification state, hashes password and issues email OTP.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

3564 @app.route('/register', methods=['POST'])
3565 def register():
3566     if is_rate_limited('register', limit=6, window_seconds=600):
3567         flash('Too many signup attempts. Please try again later.',
3568             'error')
3569         return redirect(url_for('auth_page', mode='signup'))
3570     clear_user_session_state()
3571     clear_pending_verification_state()
3572     full_name = (request.form.get('full_name') or
3573         request.form.get('fullname') or '').strip()
3574     requested_username =
3575     normalize_username(request.form.get('username', ''))
3576     email = normalize_email(request.form.get('email', ''))
3577     password = request.form.get('password', '')
3578     confirm_password = request.form.get('confirm_password', '')
3579     location = request.form.get('location', '').strip()
3580     if not full_name or not email or not password or not
3581     confirm_password or not location:
3582         flash('All fields are required!', 'error')

```

```

3579         return redirect(url_for('auth_page', mode='signup'))
3580
3581     email_error = email_validation_error(email)
3582     if email_error:
3583         flash(email_error, 'error')
3584         return redirect(url_for('auth_page', mode='signup'))
3585
3586     if password != confirm_password:
3587         flash('Passwords do not match!', 'error')
3588         return redirect(url_for('auth_page', mode='signup'))
3589
3590     if len(password) < 8:
3591         flash('Password must be at least 8 characters.', 'error')
3592         return redirect(url_for('auth_page', mode='signup'))
3593

```

Explanation: The user registration module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.3 Login Authentication

Module Name	Login Authentication
File Name	app.py
Purpose	Authenticates verified users and stores safe session values.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.

Database Interaction	The route checks password hash, blocked/deleted states and then creates user session data.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

3722 @app.route('/login', methods=['POST'])
3723 def login():
3724     if is_rate_limited('user_login', limit=10,
window_seconds=300):
3725         flash('Too many login attempts. Please try again later.',
'error')
3726         return redirect(url_for('auth_page'))
3727     clear_user_session_state()
3728     email = normalize_email(request.form.get('email', ''))
3729     password = request.form.get('password', '')
3730     remember_me = request.form.get('remember_me') == 'on'
3731
3732     if not email or not password:
3733         flash('Email and password are required!', 'error')
3734         return redirect(url_for('auth_page'))
3735     email_error = email_validation_error(email)
3736     if email_error:
3737         flash(email_error, 'error')
3738         return redirect(url_for('auth_page'))
3739
3740     conn = get_db_connection()
3741     if not conn:
3742         flash('Database connection failed.', 'error')
3743         return redirect(url_for('auth_page'))
3744
3745     try:
3746         with conn.cursor() as cursor:
3747             ensure_user_public_profile_columns(cursor)
3748             if column_exists(cursor, 'users', 'is_blocked'):
3749                 cursor.execute(
3750                     """

```

```
3751 SELECT id, username, email, password_hash,
is_blocked,
```

Explanation: The login authentication module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.4 OTP Verification

Module Name	OTP Verification
File Name	app.py
Purpose	Verifies signup OTP and activates the user account.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	OTP values are read from the database and checked with expiry before marking the user verified.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```
3193 @app.route('/verify-email', methods=['GET', 'POST'])
3194 def verify_email_page():
```

```

3195     pending_email = normalize_email(request.form.get('email') or
request.args.get('email') or session.get('...
3196     pending_user_id = session.get('pending_verification_user_id')
3197     otp_expires_at = None
3198     cooldown_remaining = 0
3199
3200     if not pending_email:
3201         flash('Please sign up or login to verify your email.',
'error')
3202         return redirect(url_for('auth_page'))
3203
3204     conn = get_db_connection()
3205     if not conn:
3206         flash('Database connection failed.', 'error')
3207         return redirect(url_for('auth_page'))
3208
3209     try:
3210         with conn.cursor() as cursor:
3211             ensure_user_public_profile_columns(cursor)
3212             ensure_email_security_schema(cursor)
3213             cursor.execute(
3214                 """
3215                 SELECT id, username, email, COALESCE(is_verified,
FALSE) AS is_verified,
3216                 verification_otp, verification_expiry,
verification_last_sent_at,
3217                 COALESCE(is_blocked, FALSE) AS is_blocked,
3218                 COALESCE(is_deleted, FALSE) AS is_deleted
3219                 FROM users
3220                 WHERE LOWER(email) = LOWER(%s)
3221                 AND (%s IS NULL OR id = %s)
3222                 ORDER BY id DESC

```

Explanation: The otp verification module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.5 Forgot Password

Module Name	Forgot Password
-------------	-----------------

File Name	app.py
Purpose	Starts password reset by sending OTP to the registered email.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The route validates email, stores a reset code and protects the flow with expiry checks.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

3360 @app.route('/forgot-password', methods=['GET', 'POST'])
3361 def forgot_password_page():
3362     reset_email = ''
3363     show_otp_form = False
3364     show_password_form = False
3365     otp_expires_at = None
3366
3367     if request.method == 'POST':
3368         action = request.form.get('action', 'send_otp')
3369         email = request.form.get('email', '').strip().lower()
3370
3371         if not email:
3372             flash('Registered email is required.', 'error')
3373             return redirect(url_for('forgot_password_page'))
3374
3375         conn = get_db_connection()
3376         if not conn:
3377             flash('Database connection failed.', 'error')

```

```

3378         return redirect(url_for('forgot_password_page'))
3379
3380     try:
3381         with conn.cursor() as cursor:
3382             ensure_user_public_profile_columns(cursor)
3383             cursor.execute(
3384                 """
3385                 SELECT id, email, reset_code,
reset_code_expiry,
3386                     COALESCE(is_blocked, FALSE) AS
is_blocked,
3387                     COALESCE(is_deleted, FALSE) AS
is_deleted
3388                 FROM users
3389                 WHERE LOWER(email) = LOWER(%s)

```

Explanation: The forgot password module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.6 Reset Password

Module Name	Reset Password
File Name	app.py
Purpose	Updates user password only after a valid reset OTP flow.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.

Database Interaction	Password reset uses a new hashed password and clears temporary reset code data.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

3430             "UPDATE users SET reset_code = NULL,
reset_code_expiry = NULL WHERE id = %s",
3431             (user['id'],)
3432         )
3433         conn.commit()
3434         flash('Email could not be sent. Please try
again later or contact support.', 'error')
3435         return render_template(
3436             'user/forgot_password.html',
3437             reset_email=user['email'],
3438             show_otp_form=False,
3439             show_password_form=False
3440         )
3441
3442         if action == 'verify_otp':
3443             reset_code = request.form.get('reset_code',
('').strip()
3444
3445             if not reset_code:
3446                 flash('OTP is required.', 'error')
3447                 return render_template(
3448                     'user/forgot_password.html',
3449                     reset_email=email,
3450                     show_otp_form=True,
3451                     show_password_form=False,
3452                     otp_expires_at=otp_expiry_timestamp(user.get('reset_code_expiry'))
3453                 )
3454
3455             cursor.execute(
3456                 """
3457                 SELECT id, email
FROM users

```

```

3458 WHERE id = %s
3459 AND LOWER(email) = LOWER(%s)

```

Explanation: The reset password module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.7 User Dashboard Route

Module Name	User Dashboard Route
File Name	app.py
Purpose	Loads dashboard statistics, engagement values and recommended users.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The dashboard checks the logged-in user before loading requests, rewards and unread counts.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

3830 @app.route('/dashboard')
3831 def dashboard_page():
3832     user = get_current_user()
3833     if not user:
3834         flash('Please log in to access the dashboard.', 'error')
3835         return redirect(url_for('auth_page'))
3836
3837     conn = get_db_connection()
3838     recommended_users = []
3839     stats = {'sent': 0, 'received': 0, 'pending': 0, 'accepted':
0, 'rejected': 0}
3840     engagement = {
3841         'badges': [],
3842         'leaderboard': [],
3843         'activity': [],
3844         'daily_users': [],
3845         'saved_users': [],
3846         'unread_notifications': 0,
3847         'unread_messages': 0,
3848         'can_claim_reward': False,
3849         'premium_active': False,
3850         'categories': []
3851     }
3852     if conn:
3853         try:
3854             with conn.cursor() as cursor:
3855                 ensure_engagement_schema(cursor)
3856                 ensure_premium_schema(cursor)
3857                 update_daily_activity(cursor, user['id'])
3858                 recommended_users, _ =
fetch_recommended_users(cursor, user, offset=0, limit=4)
3859

```

Explanation: The user dashboard route module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.8 Skill Category Listing

Module Name	Skill Category Listing
-------------	------------------------

File Name	app.py
Purpose	Displays skill categories and related users from database records.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The route loads categories, keywords and selected category users through MySQL queries.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

4184 @app.route('/skill-categories')
4185 def skill_categories_page():
4186     user = get_current_user()
4187     if not user:
4188         flash('Please log in to explore skill categories.',
4189             'error')
4189         return redirect(url_for('auth_page'))
4190
4191     categories = []
4192     selected_category = None
4193     related_users = []
4194     selected_id = request.args.get('category_id')
4195     conn = get_db_connection()
4196     if conn:
4197         try:
4198             with conn.cursor() as cursor:
4199                 categories = get_all_skill_categories(cursor)

```

```

4200             for category in categories:
4201                 keywords = [item.strip() for item in
(category.get('keywords') or '').split(',') if ite...
4202                 category['preview'] = ' / '.join(keywords[:3])
if keywords else 'Explore related skills'
4203             if selected_id:
4204                 try:
4205                     selected_id = int(selected_id)
4206                 except (TypeError, ValueError):
4207                     selected_id = None
4208             if selected_id:
4209                 selected_category = next((item for item in
categories if int(item.get('id')) == selecte...
4210                 related_users = fetch_category_users(cursor,
selected_id, limit=8)
4211             except Exception as e:
4212                 print(f"Error loading skill categories: {e}")
4213             finally:

```

Explanation: The skill category listing module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.9 Search Skills

Module Name	Search Skills
File Name	app.py
Purpose	Searches users and skills by normalized search text.
Input	Request form data, session values, URL parameters or JSON payload according to the route.

Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	Search uses query parameters and safe database filtering for user-facing results.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

4348 @app.route('/search')
4349 def search_page():
4350     user = get_current_user()
4351     if not user:
4352         flash('Please log in to access search.', 'error')
4353         return redirect(url_for('auth_page'))
4354
4355     return render_template('user/search.html',
4356                           username=user['username'], user=user, active_page='search')
4357
4358 @app.route('/profile')
4359 def profile_page():
4360     user = get_current_user()
4361     if not user:
4362         flash('Please log in to access profile.', 'error')
4363         return redirect(url_for('auth_page'))
4364
4365     avatar_presets = get_avatar_options()
4366     default_avatar = get_default_avatar_url()
4367     badges = []
4368     skill_categories = []
4369     selected_skill_category_ids = set()
4370     conn = get_db_connection()
4371     if conn:
4372         try:
4373             with conn.cursor() as cursor:
4374                 ensure_engagement_schema(cursor)
4375                 badges = get_user_badges(cursor, user['id'])

```

```

4375             skill_categories =
get_all_skill_categories(cursor)
4376             selected_skill_category_ids =
get_user_selected_category_ids(cursor, user['id'])
4377         finally:

```

Explanation: The search skills module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.10 Send Skill Request

Module Name	Send Skill Request
File Name	app.py
Purpose	Creates a pending skill swap request between two users.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The request route validates sender and receiver and prevents duplicate pending requests.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

4778 @app.route('/api/request/send', methods=['POST'])
4779 def send_request():
4780     user = get_current_user()
4781     if not user:
4782         return jsonify({'error': 'Unauthorized' }), 401
4783
4784     payload = request.get_json(silent=True) or {}
4785     receiver_id = payload.get('receiver_id')
4786     skill_requested = payload.get('skill_requested')
4787     skill_offered = payload.get('skill_offered')
4788
4789     if not receiver_id or not skill_requested or not
skill_offered:
4790         return jsonify({'error': 'receiver_id, skill_requested,
and skill_offered are required'}), 400
4791     skill_requested = str(skill_requested).strip()
4792     skill_offered = str(skill_offered).strip()
4793     if not skill_requested or not skill_offered or
len(skill_requested) > 120 or len(skill_offered) > 120:
4794         return jsonify({'error': 'Skill names must be 120
characters or less'}), 400
4795
4796     try:
4797         receiver_id = int(receiver_id)
4798     except (TypeError, ValueError):
4799         return jsonify({'error': 'Invalid receiver'}), 400
4800
4801     if receiver_id == user['id']:
4802         return jsonify({'error': 'You cannot send a request to
yourself'}), 400
4803
4804     conn = get_db_connection()
4805     if not conn:
4806         return jsonify({'error': 'Database connection failed'}),
500
4807

```

Explanation: The send skill request module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.11 Accept Request

Module Name	Accept Request
File Name	app.py
Purpose	Accepts a skill request and creates connected learning state.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The route checks request ownership before updating request status and notifying users.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

5024 @app.route('/api/request/accept', methods=['POST'])
5025 def accept_request():
5026     user = get_current_user()
5027     if not user:
5028         return jsonify({'error': 'Unauthorized'}), 401
5029
5030     payload = request.get_json(silent=True) or {}
5031     request_id = payload.get('request_id')
5032     if not request_id:
5033         return jsonify({'error': 'request_id is required'}), 400
5034
5035     conn = get_db_connection()
5036     if not conn:
5037         return jsonify({'error': 'Database connection failed'}),

```

```

500
5038
5039     try:
5040         with conn.cursor() as cursor:
5041             cursor.execute(
5042                 """
5043                 SELECT r.sender_id, r.receiver_id, r.status,
5044                 r.skill_requested, r.skill_offered,
5045                 sender.skills_offered AS sender_offers,
5046                 sender.skills_wanted AS sender_wants,
5047                 receiver.skills_offered AS receiver_offers,
5048                 receiver.skills_wanted AS receiver_wants
5049                 FROM requests r
5050                 JOIN users sender ON sender.id = r.sender_id
5051                 JOIN users receiver ON receiver.id = r.receiver_id
5052                 WHERE r.id = %s
5053                 AND r.receiver_id = %s
5054                 AND COALESCE(sender.is_deleted, FALSE) = FALSE

```

Explanation: The accept request module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.12 Reject Request

Module Name	Reject Request
File Name	app.py
Purpose	Rejects an incoming request safely.
Input	Request form data, session values, URL parameters or JSON payload according to the route.

Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The route verifies the current user and updates only the related request row.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

5130 @app.route('/api/request/reject', methods=['POST'])
5131 def reject_request():
5132     user = get_current_user()
5133     if not user:
5134         return jsonify({'error': 'Unauthorized'}), 401
5135
5136     payload = request.get_json(silent=True) or {}
5137     request_id = payload.get('request_id')
5138     if not request_id:
5139         return jsonify({'error': 'request_id is required'}), 400
5140
5141     conn = get_db_connection()
5142     if not conn:
5143         return jsonify({'error': 'Database connection failed'}),
5144         500
5145
5146     try:
5147         with conn.cursor() as cursor:
5148             ensure_unfollow_report_schema(cursor)
5149             cursor.execute(
5150                 """
5151                 SELECT r.status
5152                 FROM requests r
5153                 JOIN users sender ON sender.id = r.sender_id
5154                 WHERE r.id = %s
5155                        AND r.receiver_id = %s
5156                        AND COALESCE(sender.is_deleted, FALSE) = FALSE
5157                        AND COALESCE(sender.is_blocked, FALSE) = FALSE

```



```

5157         """
5158         (request_id, user['id'])
5159     )

```

Explanation: The reject request module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.13 Match Creation Logic

Module Name	Match Creation Logic
File Name	app.py
Purpose	Returns accepted matches with skill comparison details.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The API calculates matching skill directions and exposes safe match summaries.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

5193 @app.route('/api/matches')
5194 def get_matches():
5195     user = get_current_user()
5196     if not user:
5197         return jsonify({'error': 'Unauthorized'}), 401
5198
5199     conn = get_db_connection()
5200     if not conn:
5201         return jsonify({'error': 'Database connection failed'}),
5202         500
5203     try:
5204         with conn.cursor() as cursor:
5205             ensure_unfollow_report_schema(cursor)
5206             ensure_payment_schema(cursor)
5207             expire_chat_access(cursor)
5208             conn.commit()
5209             current_user_paid =
bool(get_user_chat_subscription(cursor, user['id']))
5210
5211             cursor.execute(
5212                 """
5213                 SELECT
5214                     m.id AS match_id,
5215                     r.id AS request_id,
5216                     COALESCE(m.skill_exchange_details,
CONCAT(r.skill_offered, ' ↔ ', r.skill_requested)...
5217                     COALESCE(m.matched_at, r.created_at) AS
matched_at,
5218                     r.status AS request_status,
5219                     r.payment_status,
5220                     %s AS current_user_paid,
5221                     IF(r.sender_id = %s, r.receiver_id,
r.sender_id) AS other_id,
5222                     u.username, u.full_name, u.location,
u.avatar_url, u.video_url, u.phone, u.email, u....

```

Explanation: The match creation logic module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.14 Chat Unlock Logic

Module Name	Chat Unlock Logic
File Name	app.py
Purpose	Creates payment order and unlocks premium chat access after success.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	Payment creation stores request and user details before gateway verification.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

5582 @app.route('/api/payment/create', methods=['POST'])
5583 def create_payment():
5584     user = get_current_user()
5585     if not user:
5586         return jsonify({'error': 'Unauthorized'}), 401
5587
5588     payload = request.get_json(silent=True) or {}
5589     request_id = payload.get('request_id')
5590
5591     if not request_id:
5592         return jsonify({'error': 'request_id is required'}), 400
5593
5594     conn = get_db_connection()
5595     if not conn:

```

```

5596         return jsonify({'error': 'Database connection failed'}),
5600
5597
5598         amount_rupees = get_chat_unlock_price_rupees()
5599         amount = int(round(amount_rupees * 100))
5600         merchant_order_id = None
5601         try:
5602             with conn.cursor() as cursor:
5603                 ensure_premium_schema(cursor)
5604                 ensure_unfollow_report_schema(cursor)
5605                 req = get_chat_request(cursor, request_id, user['id'])
5606                 conn.commit()
5607                 if not req:
5608                     return jsonify({'error': 'Chat request not
5609 found'}), 404
5609                 cursor.execute(
5610                     "SELECT id FROM unfollow_reports WHERE request_id
5611 = %s LIMIT 1",
5611                     (request_id,)

```

Explanation: The chat unlock logic module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.15 Chat Message Sending

Module Name	Chat Message Sending
File Name	app.py
Purpose	Stores chat messages and optional attachments for unlocked chats.
Input	Request form data, session values, URL parameters or JSON payload according to the route.

Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The API checks session user, request ownership and message/file validation.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

6080 @app.route('/api/chat/send', methods=['POST'])
6081 def chat_send():
6082     user = get_current_user()
6083     if not user:
6084         return jsonify({'error': 'Unauthorized'}), 401
6085
6086     is_multipart = request.content_type and
request.content_type.startswith('multipart/form-data')
6087     payload = {} if is_multipart else
(request.get_json(silent=True) or {})
6088     request_id = request.form.get('request_id') if is_multipart
else payload.get('request_id')
6089     content = ((request.form.get('content') if is_multipart else
payload.get('content')) or '').strip()
6090     receiver_id = request.form.get('receiver_id') if is_multipart
else payload.get('receiver_id')
6091     attachment = request.files.get('attachment') if is_multipart
else None
6092
6093     if not request_id or not receiver_id or (not content and not
attachment):
6094         return jsonify({'error': 'request_id, receiver_id, and
message or attachment are required'}), 400
6095     if len(content) > 2000:
6096         return jsonify({'error': 'Message must be 2000 characters
or less'}), 400
6097
6098     try:

```

```

6099         receiver_id = int(receiver_id)
6100     except (TypeError, ValueError):
6101         return jsonify({'error': 'Invalid message receiver'}), 400
6102
6103     conn = get_db_connection()
6104     if not conn:
6105         return jsonify({'error': 'Database error'}), 500
6106
6107     try:
6108         with conn.cursor() as cursor:
6109             ensure_unfollow_report_schema(cursor)

```

Explanation: The chat message sending module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.16 Payment Integration Route

Module Name	Payment Integration Route
File Name	app.py
Purpose	Handles payment verification and callback processing.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	The route verifies payment status and updates premium/chat access in the database.

Security Handling

Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

5771 @app.route('/api/payment/callback', methods=['GET', 'POST'])
5772 @app.route('/api/payment/verify', methods=['GET', 'POST'])
5773 def verify_payment():
5774     user = get_current_user()
5775     payload = request.get_json(silent=True) or {}
5776     request_id = payload.get('request_id') or
request.args.get('request_id')
5777     merchant_order_id = (
5778         payload.get('merchant_order_id')
5779         or payload.get('merchantOrderId')
5780         or payload.get('orderId')
5781         or request.args.get('merchant_order_id')
5782         or request.args.get('merchantOrderId')
5783         or request.args.get('orderId')
5784     )
5785     if not request_id and merchant_order_id and
merchant_order_id.startswith('req_'):
5786         request_id = merchant_order_id.split('_')[1]
5787     payment_user_id = user['id'] if user else None
5788
5789     try:
5790         if not request_id or not merchant_order_id:
5791             raise ValueError('Missing PhonePe payment reference')
5792
5793         print("[PhonePe Verify Debug]", {
5794             'logged_in_user_id': payment_user_id,
5795             'request_id': request_id,
5796             'merchant_order_id': merchant_order_id,
5797             'method': request.method,
5798             'args': request.args.to_dict(),
5799         })
5800

```

Explanation: The payment integration route module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business

logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.17 Payment Success Route

Module Name	Payment Success Route
File Name	app.py
Purpose	Finalizes successful payment state for chat unlock.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	Payment verification updates payment rows and creates user notifications.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

5772 @app.route('/api/payment/verify', methods=['GET', 'POST'])
5773 def verify_payment():
5774     user = get_current_user()
5775     payload = request.get_json(silent=True) or {}
5776     request_id = payload.get('request_id') or
request.args.get('request_id')
5777     merchant_order_id = (
5778         payload.get('merchant_order_id')
5779         or payload.get('merchantOrderId')

```



```

5780         or payload.get('orderId')
5781         or request.args.get('merchant_order_id')
5782         or request.args.get('merchantOrderId')
5783         or request.args.get('orderId')
5784     )
5785     if not request_id and merchant_order_id and
merchant_order_id.startswith('req_'):
5786         request_id = merchant_order_id.split('_')[1]
5787     payment_user_id = user['id'] if user else None
5788
5789     try:
5790         if not request_id or not merchant_order_id:
5791             raise ValueError('Missing PhonePe payment reference')
5792
5793         print("[PhonePe Verify Debug]", {
5794             'logged_in_user_id': payment_user_id,
5795             'request_id': request_id,
5796             'merchant_order_id': merchant_order_id,
5797             'method': request.method,
5798             'args': request.args.to_dict(),
5799         })
5800
5801         if not payment_user_id:

```

Explanation: The payment success route module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.18 Admin Login

Module Name	Admin Login
File Name	app.py
Purpose	Authenticates admin user and creates admin session.

Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	Admin login validates credentials and redirects only authorized admins to dashboard.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

6713 @app.route('/admin', methods=['GET', 'POST'])
6714 @app.route('/admin/login', methods=['GET', 'POST'])
6715 def admin_login_page():
6716     if get_current_admin():
6717         return redirect(url_for('admin_dashboard_page'))
6718
6719     if request.method == 'POST':
6720         if is_rate_limited('admin_login', limit=8,
window_seconds=600):
6721             flash('Too many login attempts. Please try again
later.', 'error')
6722             return redirect(url_for('admin_login_page'))
6723         username = request.form.get('username', '').strip()
6724         password = request.form.get('password', '').strip()
6725
6726         if not username or not password:
6727             flash('Invalid username or password', 'error')
6728             return redirect(url_for('admin_login_page'))
6729
6730         conn = get_db_connection()
6731         if conn:
6732             try:
6733                 with conn.cursor() as cursor:
6734                     ensure_admin_schema(cursor)

```

```

6735             admin_account = get_admin_account(cursor,
username=username)
6736             config = get_admin_config(cursor)
6737             conn.commit()
6738             if admin_account and
check_password_hash(admin_account['password'], password):
6739                 start_admin_session(admin_account['username'],
admin_id=0, full_name='Admin')
6740             record_admin_activity(admin_name=admin_account['username'],
action_type='admin_login', ...
6741             return
redirect(url_for('admin_dashboard_page'))
6742             if username == config['admin_username'] and
check_password_hash(config['admin_password_hash...

```

Explanation: The admin login module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.19 Manage Users

Module Name	Manage Users
File Name	app.py
Purpose	Loads admin user list and supports moderation actions.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.

Database Interaction	Admin routes use protected access and update user status through controlled actions.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

7071 @app.route('/admin/users')
7072 def manage_users_page():
7073     admin, response = admin_required()
7074     if response:
7075         return response
7076
7077     users = []
7078     conn = get_db_connection()
7079     if conn:
7080         try:
7081             with conn.cursor() as cursor:
7082                 ensure_admin_schema(cursor)
7083                 ensure_user_public_profile_columns(cursor)
7084                 ensure_premium_schema(cursor)
7085                 conn.commit()
7086                 cursor.execute(
7087                     """
7088                     SELECT id, username, full_name, email,
location, avatar_url, skills_offered,
7089                         skills_wanted, is_admin, is_blocked,
7090                         COALESCE(is_premium, FALSE) AS
is_premium,
7091                         premium_expiry_date,
7092                         COALESCE(email_notifications, TRUE) AS
email_notifications,
7093                         COALESCE(match_notifications, TRUE) AS
match_notifications,
7094                         COALESCE(profile_visibility, TRUE) AS
profile_visibility,
7095                         COALESCE(is_verified, TRUE) AS
is_verified,
7096                         COALESCE(is_deleted, FALSE) AS
is_deleted,

```

```

7097                                     block_reason, delete_reason,
deleted_at, created_at,
7098                                     (
7099                                     SELECT aa.action_type
7100                                     FROM admin_actions aa

```

Explanation: The manage users module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.20 Manage Skills

Module Name	Manage Skills
File Name	app.py
Purpose	Manages skill categories from the admin panel.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	Admin skill routes save, update and delete category records using safe form values.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

7483 @app.route('/admin/skills')
7484 def admin_skills_page():
7485     admin, response = admin_required()
7486     if response:
7487         return response
7488
7489     skills = []
7490     categories = []
7491     conn = get_db_connection()
7492     if conn:
7493         try:
7494             with conn.cursor() as cursor:
7495                 ensure_engagement_schema(cursor)
7496                 cursor.execute(
7497                     """
7498                     SELECT u.id, u.username, u.full_name, u.email,
7499                     u.skills_offered AS teach_skill,
7500                     u.skills_wanted AS learn_skill,
7501                     u.created_at,
7502                     GROUP_CONCAT(sc.category_name ORDER BY
7503                     sc.category_name SEPARATOR ', ') AS categ...
7504                     FROM users u
7505                     LEFT JOIN user_skill_categories usc ON
7506                     usc.user_id = u.id
7507                     LEFT JOIN skill_categories sc ON sc.id =
7508                     usc.category_id
7509                     WHERE (u.skills_offered IS NOT NULL AND
7510                     u.skills_offered != '')
7511                     OR (u.skills_wanted IS NOT NULL AND
7512                     u.skills_wanted != '')
7513                     OR usc.category_id IS NOT NULL
7514                     GROUP BY u.id, u.username, u.full_name,
7515                     u.email, u.skills_offered, u.skills_wanted, u.c...
7516                     ORDER BY u.created_at DESC
7517                     """
7518                 )
7519             skills = cursor.fetchall()
7520             categories = get_all_skill_categories(cursor)

```

Explanation: The manage skills module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.21 Reports and Analytics

Module Name	Reports and Analytics
File Name	app.py
Purpose	Displays user reports and moderation data for admin review.
Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	Reports page reads report records and supports moderation action routes.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

8388 @app.route('/admin/reports')
8389 def admin_reports_page():
8390     admin, response = admin_required()
8391     if response:
8392         return response
8393
8394     reports = []
8395     report_stats = {
8396         'users_by_location': [],
8397         'requests_by_status': [],
8398         'payments_by_status': [],
8399     }

```

```

8400     conn = get_db_connection()
8401     if conn:
8402         try:
8403             with conn.cursor() as cursor:
8404                 ensure_admin_schema(cursor)
8405                 conn.commit()
8406                 cursor.execute(
8407                     """
8408                     SELECT r.id, r.reason, r.description,
8409                     r.status, r.created_at,
8410                     COALESCE(r.reporter_username,
8411                     reporter.username, CONCAT('User #', COALESCE(r.rep...
8412                     COALESCE(r.reported_username,
8413                     reported.username, CONCAT('User #', r.reported_use...
8414                     r.reported_user_id
8415                     FROM reports r
8416                     LEFT JOIN users reporter ON reporter.id =
8417                     COALESCE(r.reported_by, r.reporter_user_id)
8418                     LEFT JOIN users reported ON reported.id =
8419                     r.reported_user_id
8420                     ORDER BY r.created_at DESC
8421                     """
8422                 )

```

Explanation: The reports and analytics module is written using Python Flask syntax and fits the SkillFlow stack. It does not use PHP include statements, PHP variables, or PHP database calls. The route or helper receives controlled input from Flask request/session objects, validates the data, executes MySQL work with cursor.execute where required, and returns a clean response. This keeps business logic on the server side and makes the implementation safer for authentication, matching, payment, chat, notification and admin workflows.

9.22 Notifications and Activity Logs

Module Name	Notifications and Activity Logs
File Name	app.py
Purpose	Stores notifications and admin audit history.

Input	Request form data, session values, URL parameters or JSON payload according to the route.
Output	Rendered template, redirect response, JSON response, notification, or database update.
Database Interaction	Notification helpers insert safe messages while activity logs track admin actions.
Security Handling	Session validation, normalized inputs, password hashing, rate limiting, ownership checks, and safe parameterized cursor.execute queries are used where required.

Line-numbered Python Flask code:

```

2183 def create_user_notification(cursor, user_id, notification_type,
2184 title, message, related_id=None):
2185     ensure_engagement_schema(cursor)
2186     cursor.execute(
2187         """
2188         INSERT INTO user_notifications (user_id,
2189 notification_type, title, message, related_id)
2190         VALUES (%s, %s, %s, %s, %s)
2191         """,
2192         (user_id, notification_type, title, message, related_id)
2193     )
2194
2195 def award_achievement(cursor, user_id, badge_key):
2196     ensure_engagement_schema(cursor)
2197     badge = ENGAGEMENT_BADGES.get(badge_key)
2198     if not badge:
2199         return
2200     cursor.execute(
2201         """
2202         INSERT IGNORE INTO user_achievements (user_id, badge_key,
2203 badge_name, icon)
2204         VALUES (%s, %s,

```

CHAPTER 10 - VALIDATION CHECKS

Form Validation

Form Validation in SkillFlow represents entity relationships among users, skills, requests, matches, payments, messages and reports so that database dependency can be understood clearly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Area	Project-Specific Explanation
Objective	Objective is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Input	Input is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Expected Result	Expected Result is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Observation	Observation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
-------------	--

Login Validation

Login Validation in SkillFlow documents user behaviour through UML style views that make login, profile editing, request sending, chat access and admin review easier to explain during viva.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Signup Validation

Signup Validation in SkillFlow describes database design with attention to field purpose, relationship meaning, integrity and the way MySQL supports persistent project records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Payment Validation

Payment Validation in SkillFlow explains each module by connecting the page seen by the user with the route, validation checks and table operations performed in the backend.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

List Validation

Request List Validation in SkillFlow presents authentication as the first controlled entry point where user identity, session values and validation rules protect the rest of the application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Skill Validation

Skill Validation in SkillFlow explains OTP verification as an additional confirmation step that improves trust before sensitive account actions are accepted by the system.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Admin Validation

Admin Validation in SkillFlow studies skill matching as the main business logic where offered skills, required skills and user interests are compared to suggest meaningful exchange opportunities.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Server Side Validation

Server Side Validation in SkillFlow explains the chat system as a controlled communication area that becomes useful after the system has enough information to connect two users responsibly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Database Validation

Database Validation in SkillFlow discusses payment integration through plan selection, transaction recording and administrative visibility rather than treating payment as a separate isolated feature.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Upload Validation

Upload Validation in SkillFlow describes the dashboard as the user's working area where personal details, skills, requests, matches, notifications and recent activity are brought together.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

CHAPTER 11 - TESTING

Unit Testing

Unit Testing in SkillFlow presents the admin panel as the management side of the software where platform activity can be supervised through users, reports, plans, analytics and notifications.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Area	Project-Specific Explanation
Objective	Objective is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Input	Input is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Expected Result	Expected Result is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Observation	Observation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Integration Testing

Integration Testing in SkillFlow explains validation by showing how forms, route checks, database constraints and user feedback reduce incorrect entries and incomplete submissions.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

System Testing

System Testing in SkillFlow discusses security from the viewpoint of sessions, password handling, OTP checks, role separation, controlled routes and careful processing of user supplied values.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Black Box Testing

Black Box Testing in SkillFlow explains testing through realistic operations such as registration, login, skill posting, request creation, chat unlock, payment recording and admin review.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Security Testing

Security Testing in SkillFlow presents results by connecting completed features with observable outputs on pages, database records and admin dashboard summaries.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Validation Testing

Validation Testing in SkillFlow describes advantages through practical benefits such as organized exchange, reduced manual search, better accountability and improved learning collaboration.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Responsive Testing

Responsive Testing in SkillFlow discusses limitations honestly, including the dependency on correct user data, internet availability and future scope for stronger recommendation intelligence.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Admin Testing

Admin Testing in SkillFlow presents future scope by identifying improvements such as AI matching, mobile application support, advanced analytics, richer communication and scalable deployment.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Test ID	Module	Test Case	Input	Expected	Actual	Result
TC-001	Registration	Execute valid and invalid registration workflow	Sample data	Correct result/error	As expected	Pass
TC-002	Login	Execute valid and invalid login workflow	Sample data	Correct result/error	As expected	Pass
TC-003	Skill Listing	Execute valid and invalid skill listing workflow	Sample data	Correct result/error	As expected	Pass
TC-004	Search	Execute valid and invalid search workflow	Sample data	Correct result/error	As expected	Pass

TC-005	Request List	Execute valid and invalid request list workflow	Sample data	Correct result/error	As expected	Pass
TC-006	Chat Unlock	Execute valid and invalid chat unlock workflow	Sample data	Correct result/error	As expected	Pass
TC-007	Payment	Execute valid and invalid payment workflow	Sample data	Correct result/error	As expected	Pass
TC-008	Request Tracking	Execute valid and invalid request tracking workflow	Sample data	Correct result/error	As expected	Pass
TC-009	Admin Login	Execute valid and invalid admin login workflow	Sample data	Correct result/error	As expected	Pass
TC-010	Add Skill	Execute valid and invalid add skill workflow	Sample data	Correct result/error	As expected	Pass
TC-011	Edit Skill	Execute valid and invalid edit skill workflow	Sample data	Correct result/error	As expected	Pass
TC-012	Delete Skill	Execute valid and invalid delete skill workflow	Sample data	Correct result/error	As expected	Pass
TC-013	Manage Requests	Execute valid and invalid manage requests workflow	Sample data	Correct result/error	As expected	Pass
TC-014	Notifications	Execute valid and invalid notifications workflow	Sample data	Correct result/error	As expected	Pass

TC-015	Reports	Execute valid and invalid reports workflow	Sample data	Correct result/error	As expected	Pass
TC-016	Responsive Home	Execute valid and invalid responsive home workflow	Sample data	Correct result/error	As expected	Pass
TC-017	Profile	Execute valid and invalid profile workflow	Sample data	Correct result/error	As expected	Pass
TC-018	Reviews	Execute valid and invalid reviews workflow	Sample data	Correct result/error	As expected	Pass
TC-019	Payment History	Execute valid and invalid payment history workflow	Sample data	Correct result/error	As expected	Pass
TC-020	Inventory Check	Execute valid and invalid inventory check workflow	Sample data	Correct result/error	As expected	Pass

CHAPTER 12 - IMPLEMENTATION AND MAINTENANCE

local Flask/MySQL environment Setup

local Flask/MySQL environment Setup in SkillFlow concludes the project by summarizing how the planned learning exchange system was converted into a working Flask and MySQL based web application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Area	Project-Specific Explanation
Objective	Objective is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Input	Input is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Expected Result	Expected Result is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Observation	Observation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Database Import

Database Import in SkillFlow introduces how the platform converts informal peer learning into an organized digital workflow where users can publish skills, discover compatible learners and begin structured exchanges through a Flask powered interface.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Localhost Setup

Localhost Setup in SkillFlow explains the learning exchange idea from the user's point of view, showing how a person moves from registration to skill discovery, request creation, matching and controlled communication.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Project Folder Setup

Project Folder Setup in SkillFlow focuses on the practical need for a web based environment where students and professionals can trade knowledge without depending on offline contacts or scattered social media messages.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Configuration

Configuration in SkillFlow describes the project's core purpose: to bring skill listing, matching, request handling, chat access, notifications and administrative supervision into one connected application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Deployment Process

Deployment Process in SkillFlow presents the academic and technical goals of the project, especially the use of Python Flask, HTML, CSS, JavaScript and MySQL to solve a real collaboration problem.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Maintenance Plan

Maintenance Plan in SkillFlow defines the boundary of the application by separating learner features, admin controls, database operations, payment supported access and report handling into clear functional areas.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Backup Plan

Backup Plan in SkillFlow discusses the problem that learners often know what they want to learn but do not have a reliable way to find someone who needs their own skill in return.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Skillion Checklist

Skillion Checklist in SkillFlow shows how the development process was planned around requirement study, database design, route implementation, interface preparation, validation, testing and final deployment readiness.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

User Training

User Training in SkillFlow explains how administrative automation supports the project by monitoring users, reports, requests, payments and platform activity from a controlled backend panel.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

CHAPTER 13 - REPORTS, TABLES AND FIGURES

Sales Report

Sales Report in SkillFlow highlights the database driven nature of the system, where important actions are stored, retrieved and updated through MySQL tables instead of remaining temporary browser data.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Area	Project-Specific Explanation
Objective	Objective is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Input	Input is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Expected Result	Expected Result is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Observation	Observation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Payment Report

Payment Report in SkillFlow connects digital learning requirements with practical software engineering by combining authentication, search, matching, messaging and analytics into a single student project.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Request Report

Request Report in SkillFlow emphasizes user convenience through clean navigation, responsive pages, visible status messages, quick notifications and a dashboard that keeps major actions easy to reach.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

User Report

User Report in SkillFlow studies the existing limitations of manual learning exchange and explains why a dedicated platform gives better tracking, accountability and communication control.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Inventory Report

Inventory Report in SkillFlow presents the proposed solution as a manageable web application where every major process is represented through clear routes, templates, forms and database relationships.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Activity Report

Activity Report in SkillFlow examines technical feasibility through the availability of Flask, MySQL, browser based frontend tools and a development environment that can be maintained on standard systems.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Admin Analytics Explanation

Admin Analytics Explanation in SkillFlow evaluates operational feasibility by considering whether normal users can register, add skills, send requests, unlock chat and understand notifications without technical training.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Dashboard Output Interpretation

Dashboard Output Interpretation in SkillFlow discusses economic feasibility because the project can be built with open source technologies while still supporting premium style features such as plans and payment records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

CHAPTER 14 - SCREENSHOTS OF PROJECT

This chapter contains clean screenshot insertion pages for the final project screens. Each screenshot page has exactly one title, one placeholder box, one figure caption and one short explanation paragraph.

14.1 LOGIN PAGE

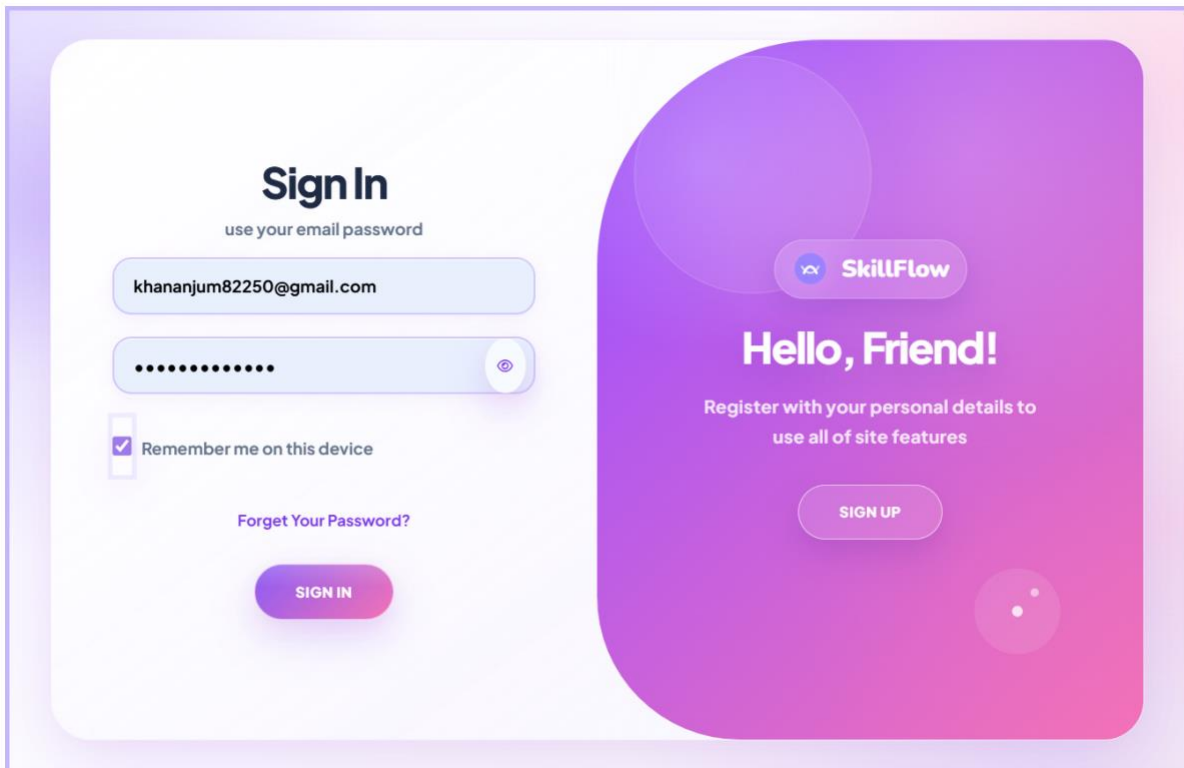
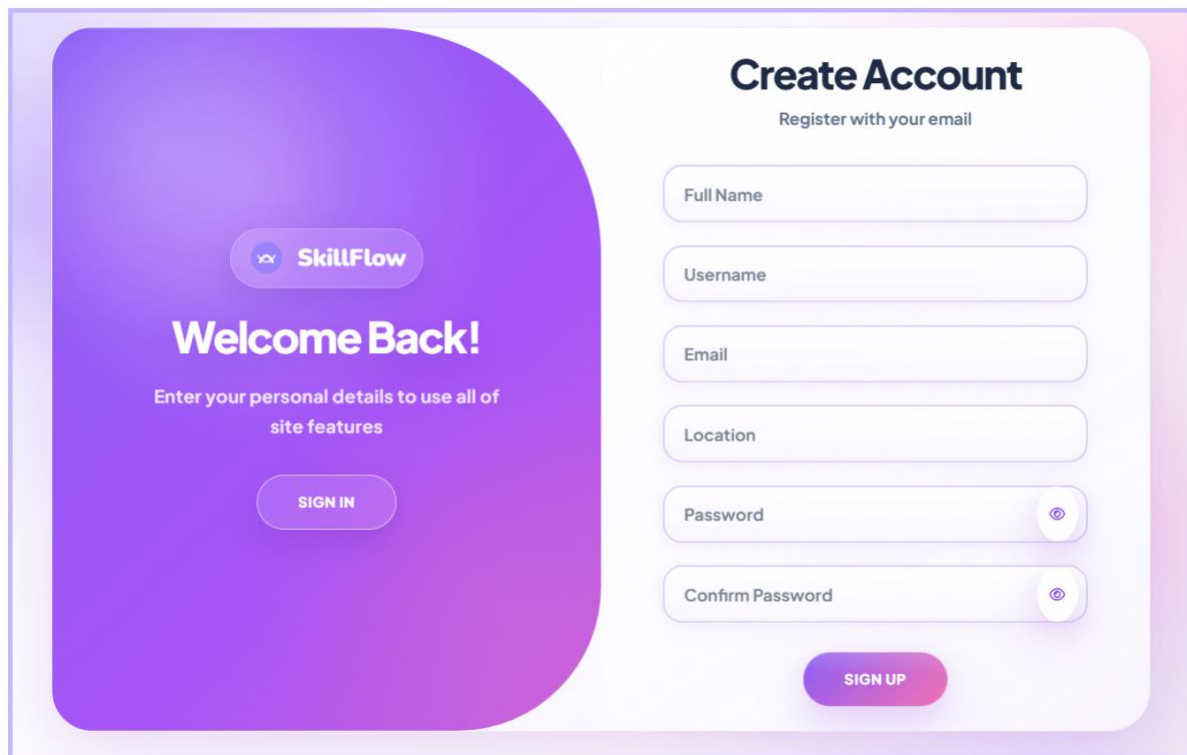


Figure 14.1: Login Page

The login page allows registered users to access SkillFlow securely. It validates user credentials and redirects authenticated users to the dashboard.

14.2 REGISTER PAGE



The image shows a web interface for creating an account on SkillFlow. On the left, a purple rounded rectangle contains the SkillFlow logo, the text 'Welcome Back!', and a 'SIGN IN' button. On the right, the 'Create Account' section prompts the user to 'Register with your email'. It includes input fields for Full Name, Username, Email, Location, Password, and Confirm Password, each with a toggle icon for visibility. A 'SIGN UP' button is at the bottom right.

Create Account
Register with your email

Full Name

Username

Email

Location

Password

Confirm Password

SIGN UP

Figure 14.2: Register Page

The register page collects user details and begins the email verification process. It is the first step for creating a SkillFlow learning profile.

14.3 OTP VERIFICATION PAGE

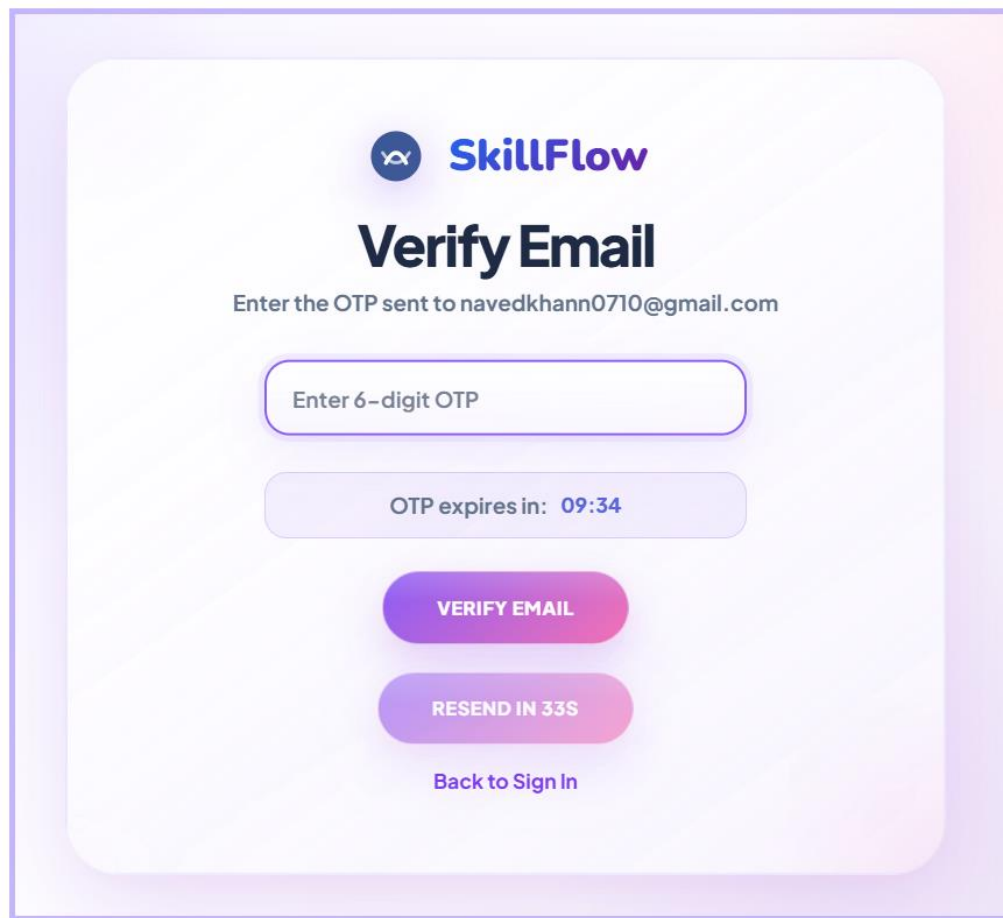


Figure 14.3: OTP Verification Page

The OTP verification page confirms the user email address. It improves account authenticity before giving access to protected modules.

14.4 DASHBOARD PAGE

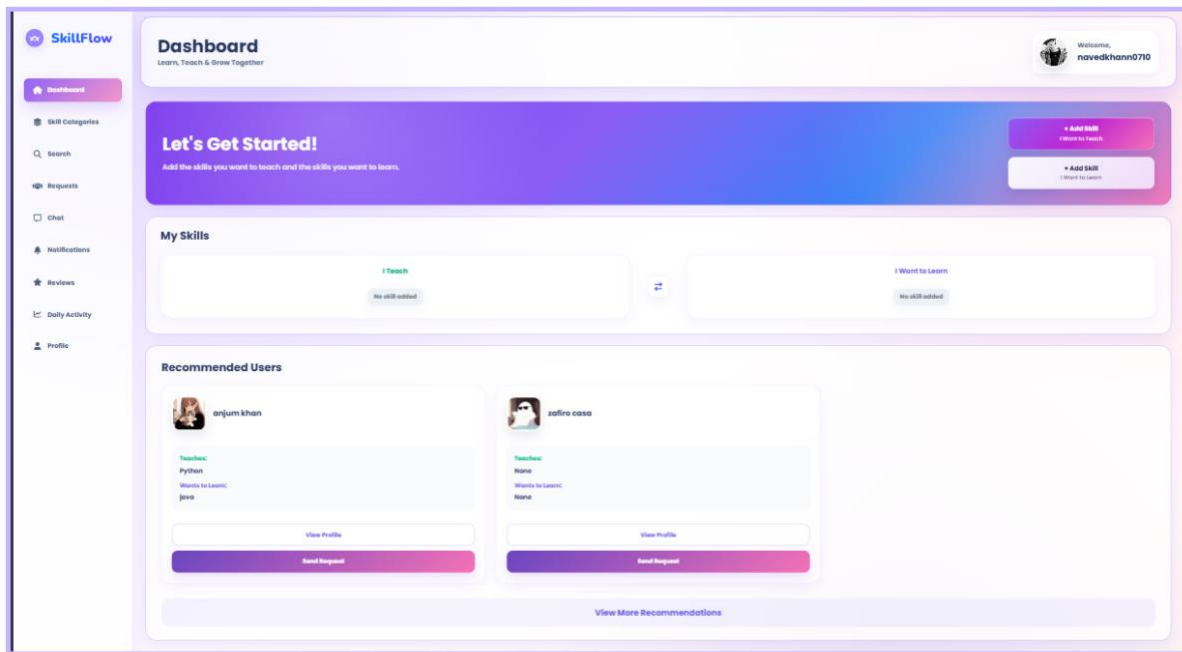
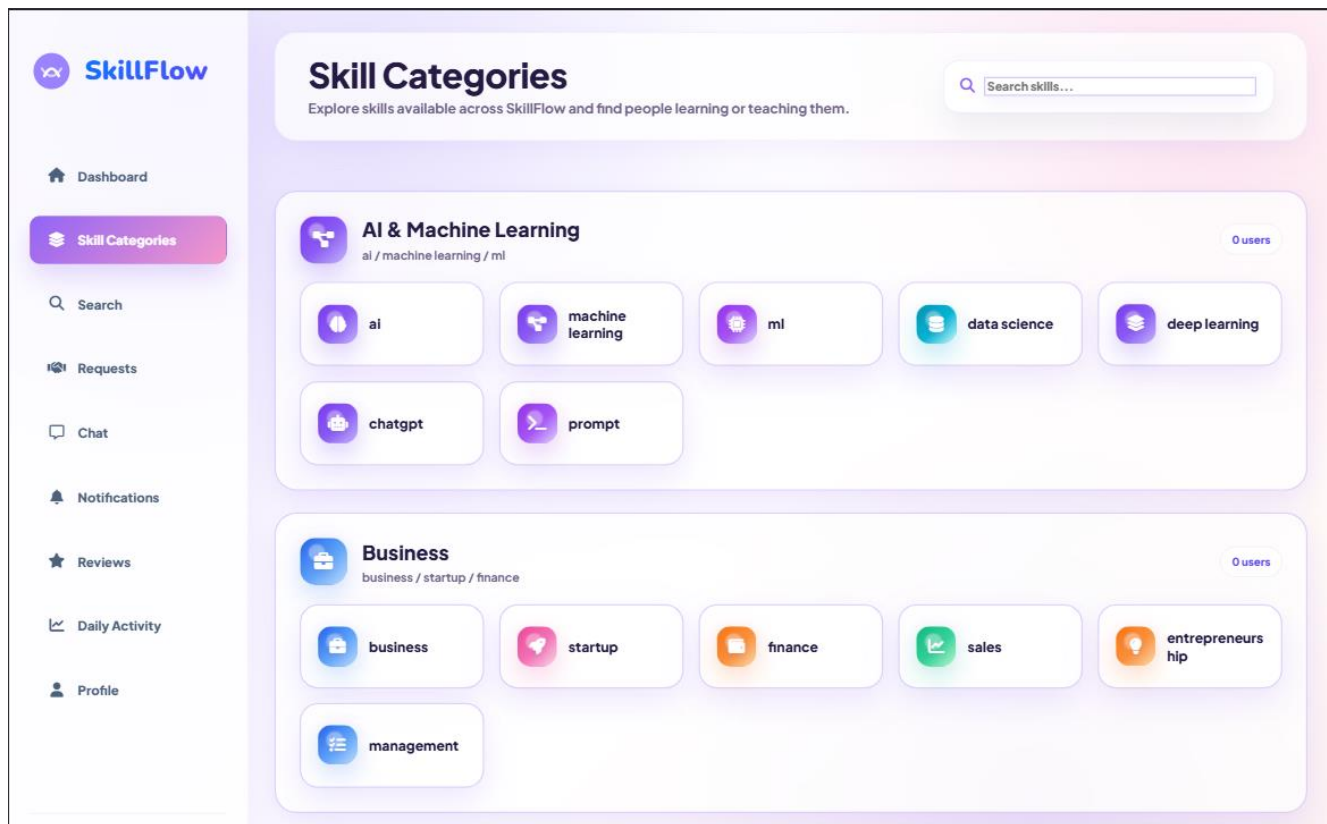


Figure 14.4: Dashboard Page

The dashboard presents user activity, skills, requests and quick actions in one organized view. It works as the main control area for learners.

14.5 SKILL CATEGORIES PAGE



14.5: Skill Categories Page

The skill categories page displays available learning areas and helps users explore relevant skills before searching for partners.

14.6 SEARCH PAGE

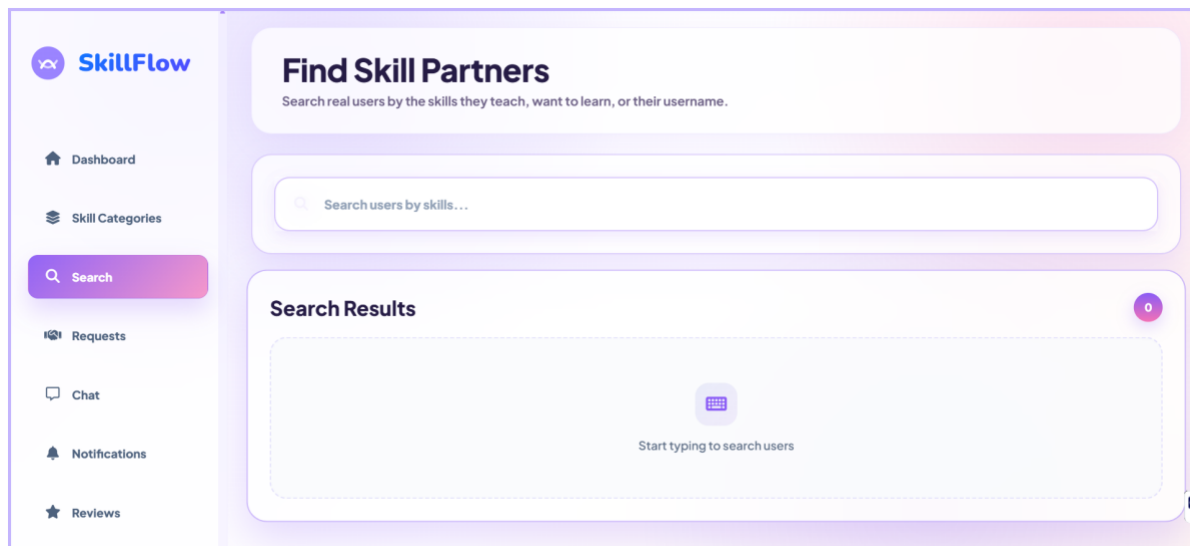


Figure 14.6: Search Page

The search page helps users find other learners by skill name, category or learning interest. Results are generated dynamically from database records.

14.7 SEARCH RESULTS PAGE

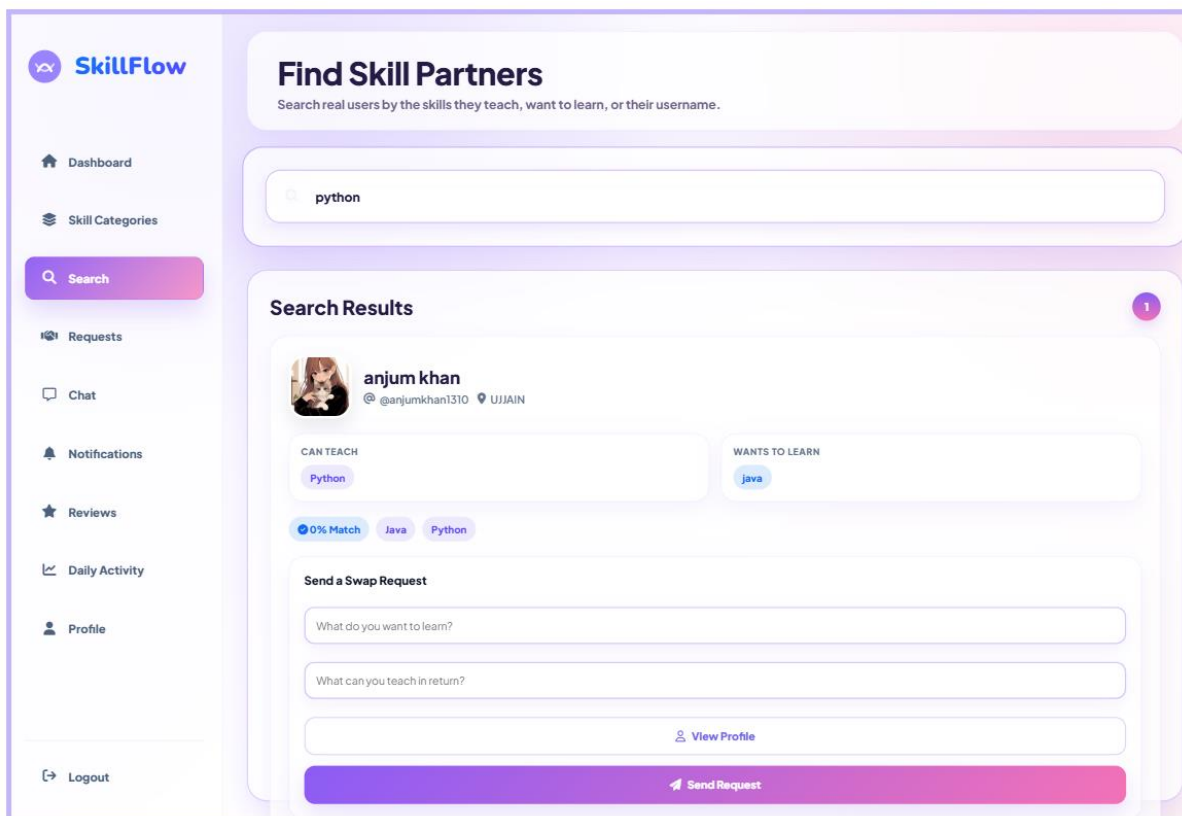


Figure 14.7: Search Results Page

The search results screen shows matching users with profile information and request options. It supports the main skill exchange workflow.

14.8 REQUESTS PAGE

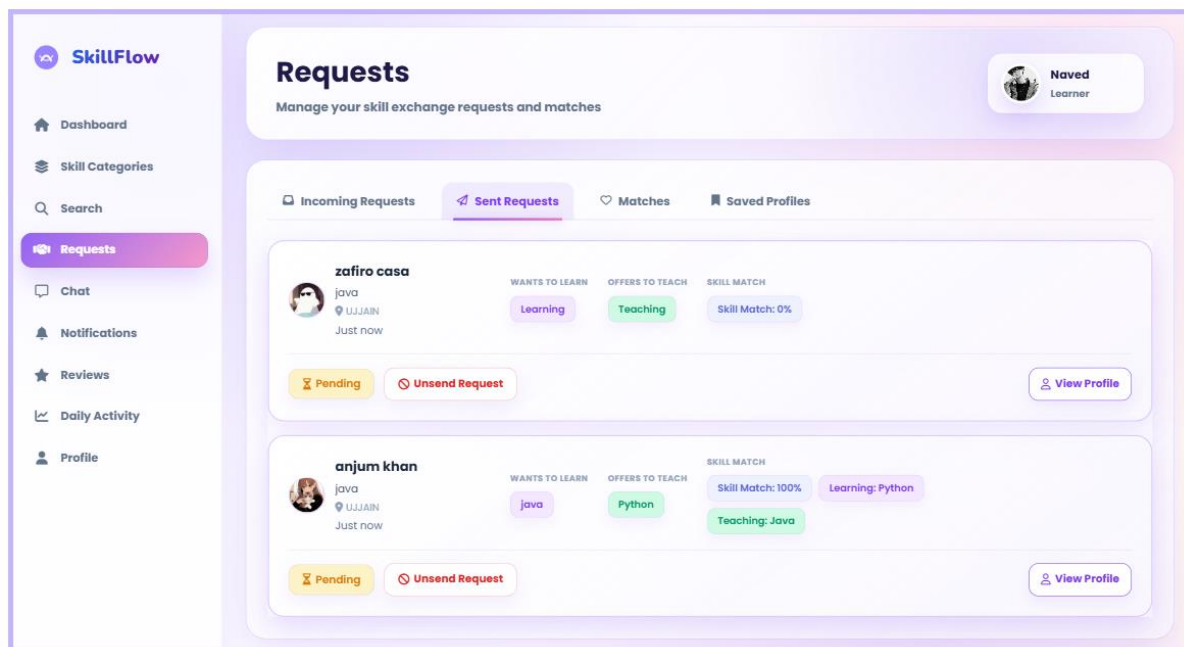


Figure 14.8: Requests Page

The requests page manages incoming, sent, matched and saved profile interactions. It keeps the exchange process clear and trackable.

14.9 MATCHES PAGE

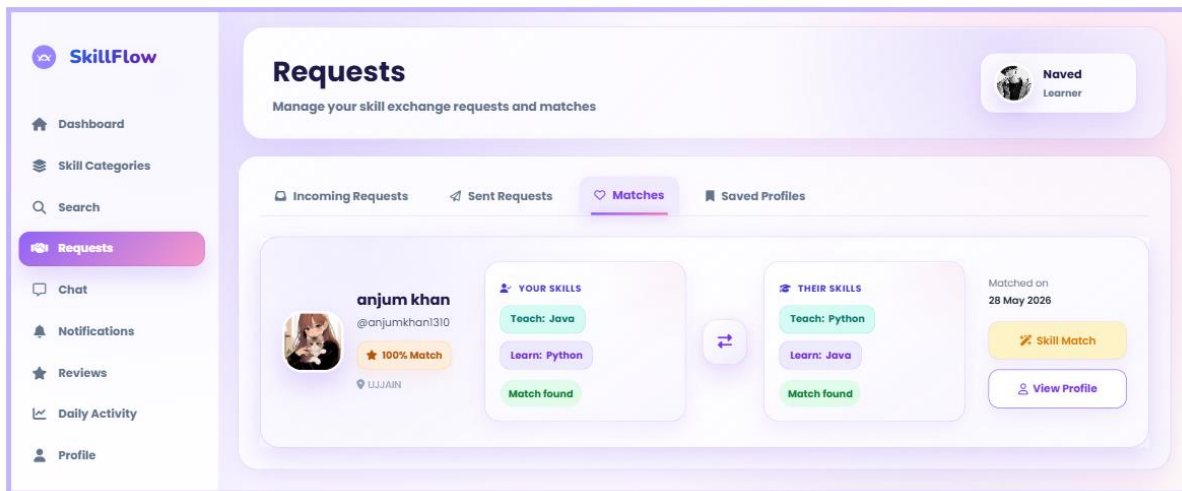


Figure 14.9: Matches Page

The matches page compares learning and teaching skills between connected users. It helps users understand why a match is useful.

14.10 CHAT UNLOCK PAGE

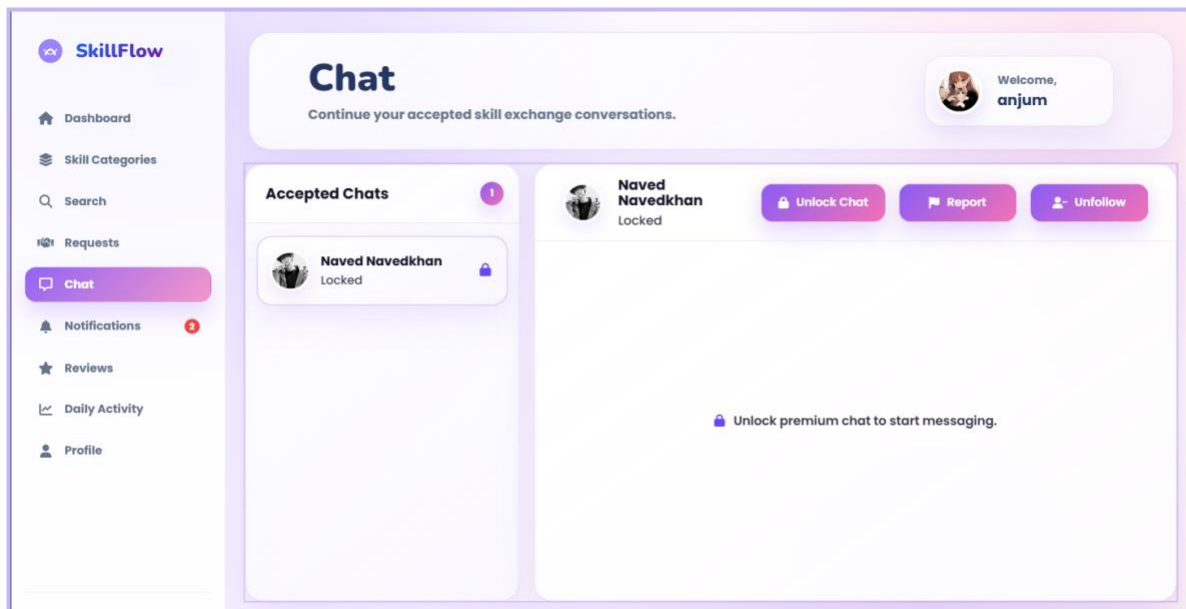


Figure 14.10: Chat Unlock Page

The chat unlock page connects accepted requests with payment-based access where required. It protects premium communication flow.

14.11 PAYMENT UNLOCK PAGE

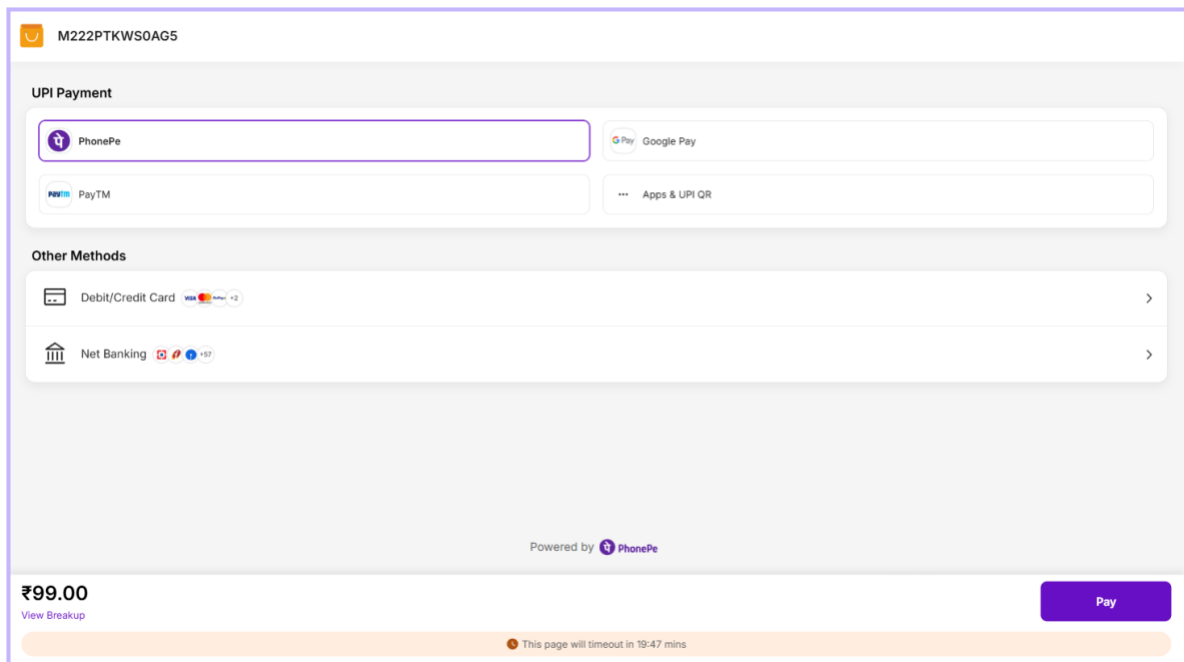


Figure 14.11: Payment Unlock Page

The payment unlock screen confirms payment progress and access status. It supports premium chat activation after successful verification.

14.12 USER PROFILE PAGE

Profile Avatar
 Choose a clear headshot avatar that matches SkillFlow.

Avatar: Username: @jenkinsun2025 Follower: 5.5K Post: 200 Premium: Active

[Change Avatar](#)

Achievement Badges
 Premium User | Skill Master

Basic Information

Full Name: Username:

Email Address: Location:

[Edit Info](#)

Skills & Expertise

Skills I Can Teach: Skills I Want to Learn:

[Edit Skills](#)

About You & Demo

Bio / Description:

Demo/Tutorial Video URL:

Demo Video Description:

[Edit Bio](#)

Contact Details (Private)

Phone Number: Alternative Contact Number:

Instagram ID:

☐ Allow sharing my contact details (phone & email) with users who have paid for a skill design with me.

Figure 14.12: User Profile Page

The profile page displays user identity, skills, learning goals and review options. It is important for trust and skill discovery.

14.13 NOTIFICATIONS PAGE

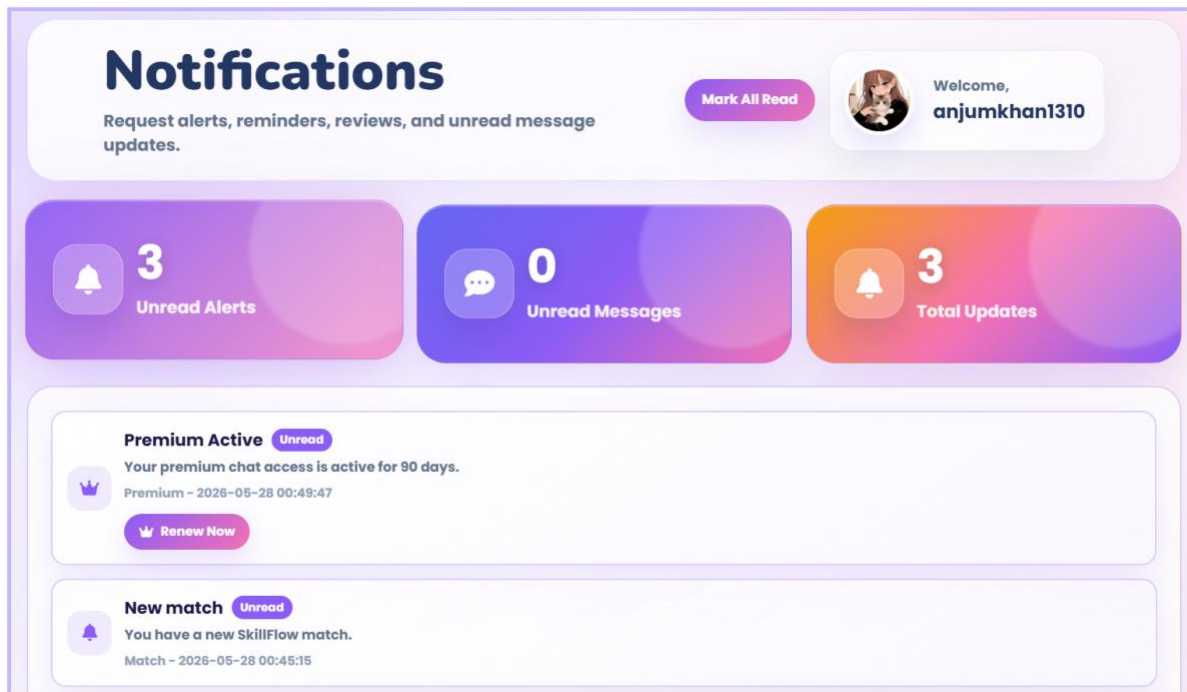


Figure 14.13: Notifications Page

The notifications page shows request alerts, premium reminders, chat updates and review messages. It keeps users informed about platform activity.

14.14 DAILY ACTIVITY PAGE

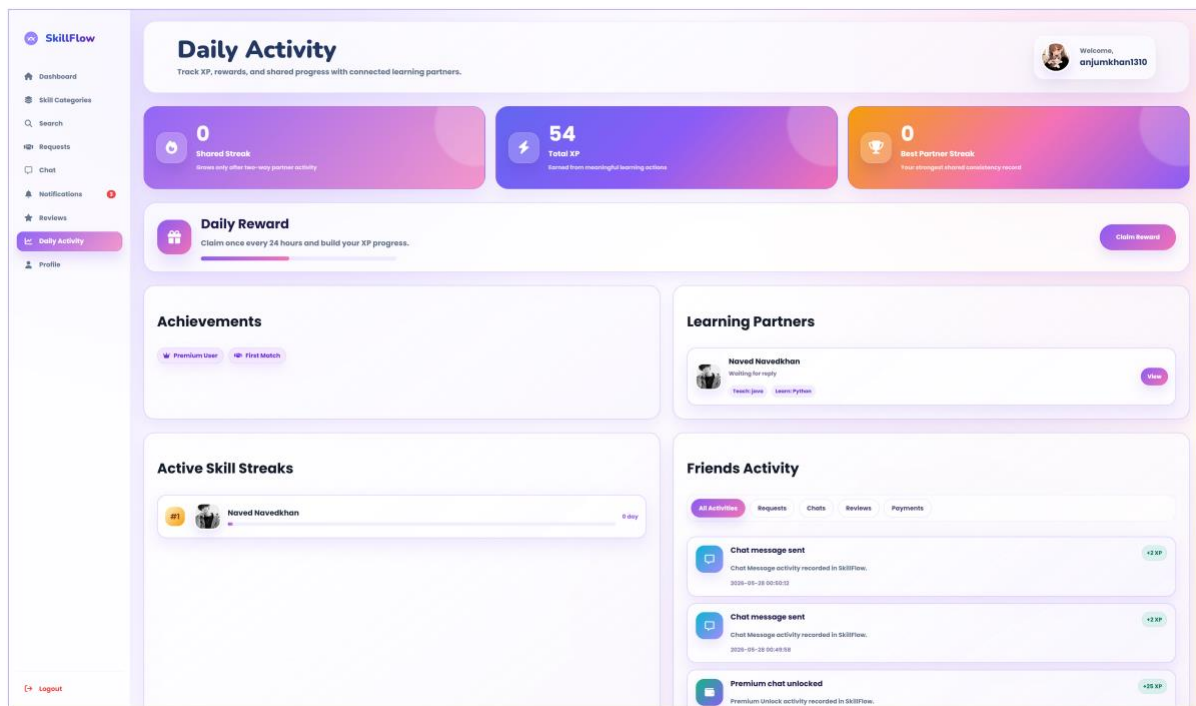
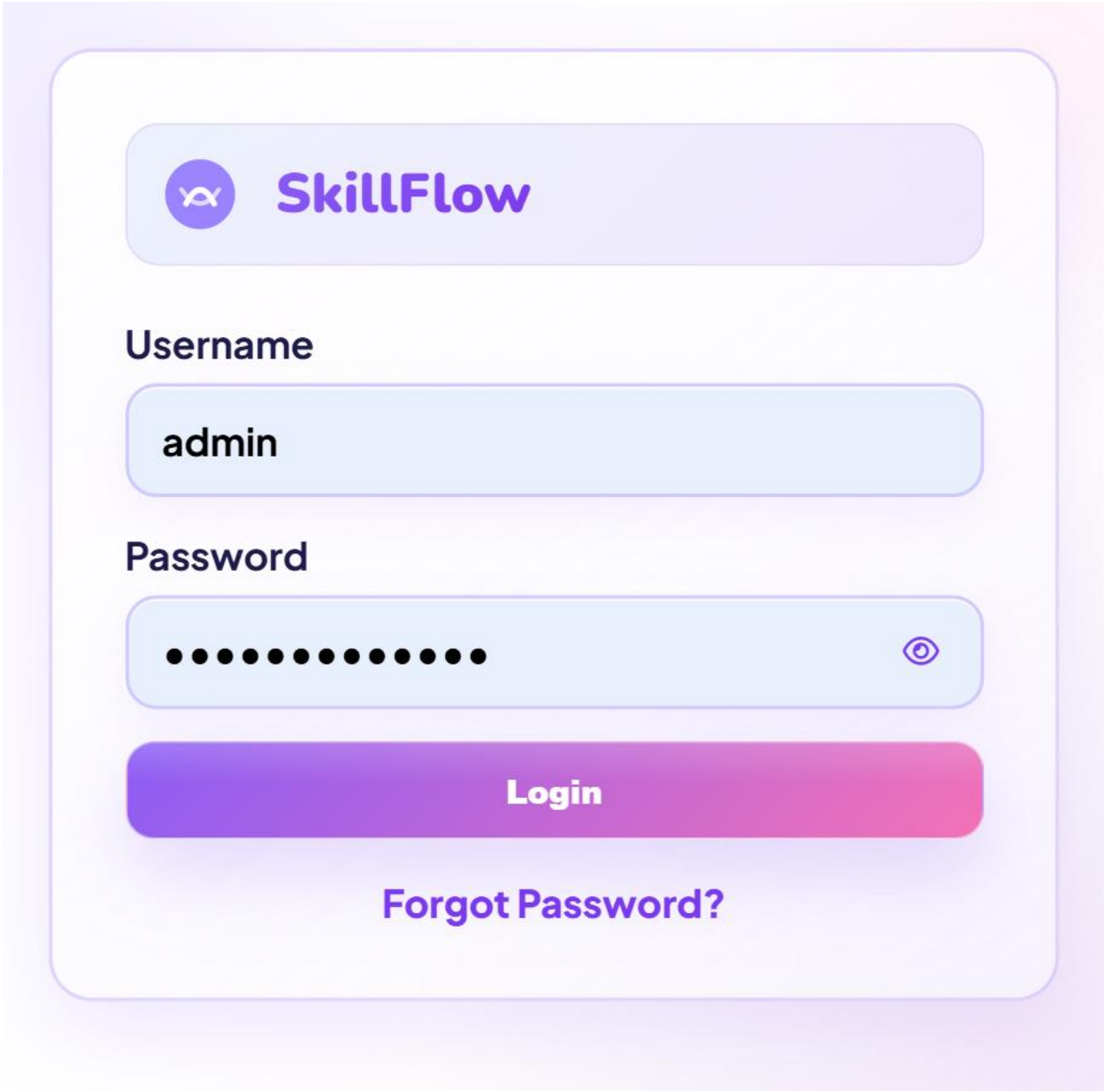


Figure 14.14: Daily Activity Page

The daily activity page tracks XP, rewards, streaks and meaningful learning activity. It encourages consistent participation.

14.15 ADMIN LOGIN PAGE



The image shows a UI mockup of an admin login page. It features a light purple background with a central white rounded rectangle containing the login form. At the top of the form is the SkillFlow logo, which consists of a purple circle with a white fish-like icon and the text 'SkillFlow' in a bold, purple, sans-serif font. Below the logo, the label 'Username' is followed by a light blue rounded input field containing the text 'admin'. The label 'Password' is followed by a similar light blue rounded input field filled with black dots, and a small purple eye icon to its right. Below these fields is a wide, rounded button with a purple-to-pink gradient and the text 'Login' in white. At the bottom of the form is the text 'Forgot Password?' in a purple, sans-serif font.

Figure 14.15: Admin Login Page

The admin login page protects administrative access. Only authorized admin users can enter the monitoring and management panel.

14.16 ADMIN DASHBOARD PAGE

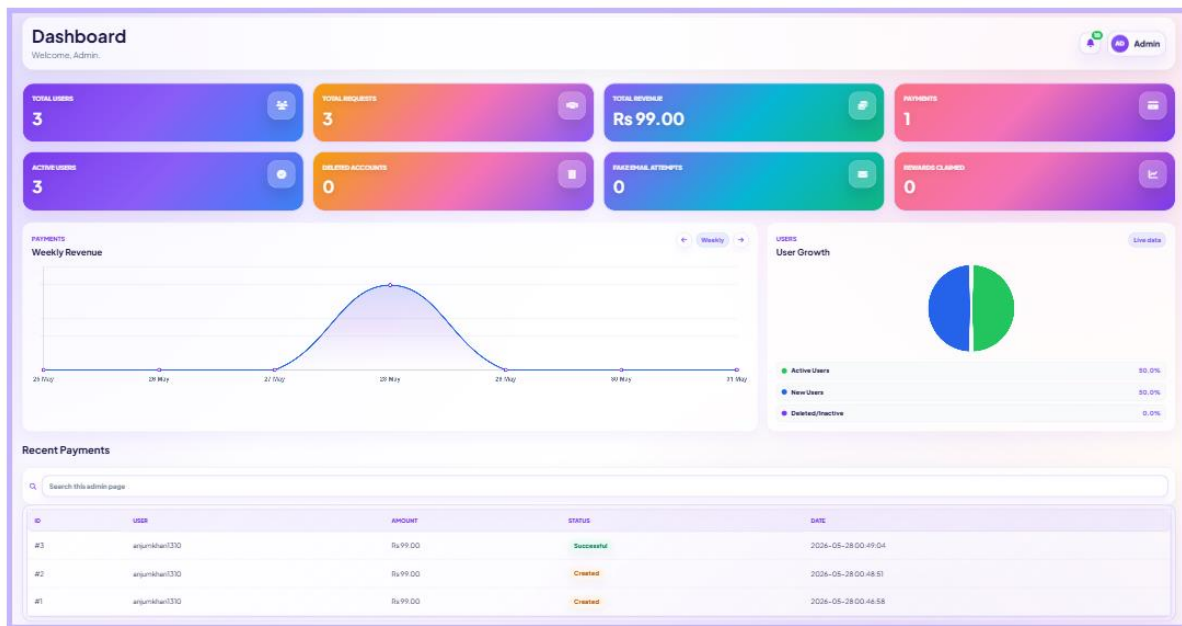


Figure 14.16: Admin Dashboard Page

The admin dashboard summarizes users, requests, payments and platform activity. It gives a quick overview of the complete system.

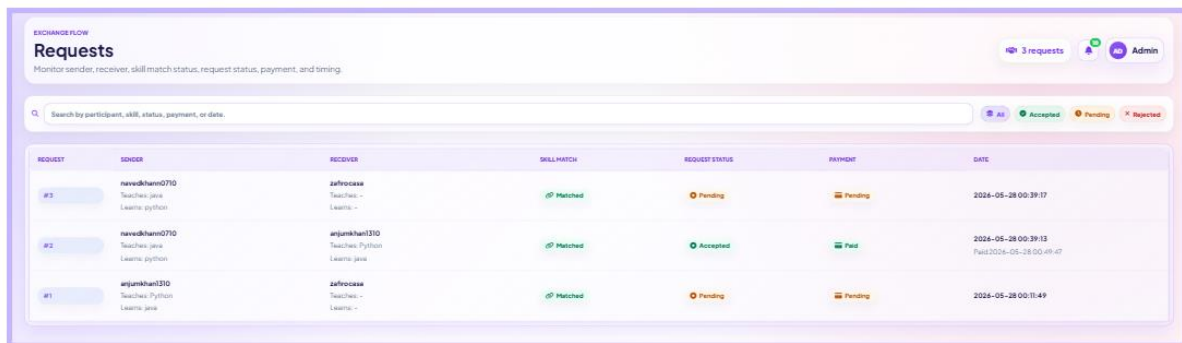
14.17 ADMIN USERS PAGE

USER	CONTACT	LOCATION	SKILLS	VERIFICATION	STATUS	MONITORING	ACTIONS
Naved Navedkhan @navednaved770 - #3	navedkhan0710@gmail.com C3334-09-28 00:16:05	📍 US/IN	🔧 Java 🔧 python	🟢 Verified	🟢 Active	🔔 Free 🔔 Email On / Match On 🔔 Public	<button>Block</button> <button>Unblock</button>
anjum khan @anjumkhan010 - #14	khanumkhan0220@gmail.com C3334-09-28 00:04:27	📍 US/IN	🔧 Python 🔧 Java	🟢 Verified	🟡 Logged Out	🔔 Premium 🔔 Email On / Match On 🔔 Public	<button>Block</button> <button>Unblock</button>
zaPro casa @zaProCasa - #3	zaProCasaAdmin@gmail.com C3334-09-27 23:23:45	📍 US/IN	🔧 No offered skills 🔧 No wanted skills	🟢 Verified	🟡 Logged Out	🔔 Free 🔔 Email On / Match On 🔔 Public	<button>Block</button> <button>Unblock</button>

Figure 14.17: Admin Users Page

The admin users page supports user verification, blocking, deletion and monitoring. It helps maintain platform discipline and safety.

14.18 ADMIN REQUESTS PAGE



EXCHANGE FLOW
Requests
Monitor sender, receiver, skill match status, request status, payment, and timing.

3 requests | Admin

Search by participant, skill, status, payment, or date.

REQUEST	SENDER	RECEIVER	SKILL MATCH	REQUEST STATUS	PAYMENT	DATE
#3	navedkhan0710 Teaches: java Learns: python	zafrocasa Teaches: - Learns: -	Matched	Pending	Pending	2024-05-28 00:39:17
#2	navedkhan0710 Teaches: java Learns: python	arjunkhan0310 Teaches: Python Learns: java	Matched	Accepted	Paid	2024-05-28 00:39:13 Paid: 2024-05-28 00:49:47
#1	arjunkhan0310 Teaches: Python Learns: java	zafrocasa Teaches: - Learns: -	Matched	Pending	Pending	2024-05-28 00:11:49

Figure 14.18: Admin Requests Page

The admin requests page displays skill exchange request records and their status. It assists in reviewing platform interaction flow.

14.19 ADMIN PAYMENTS PAGE

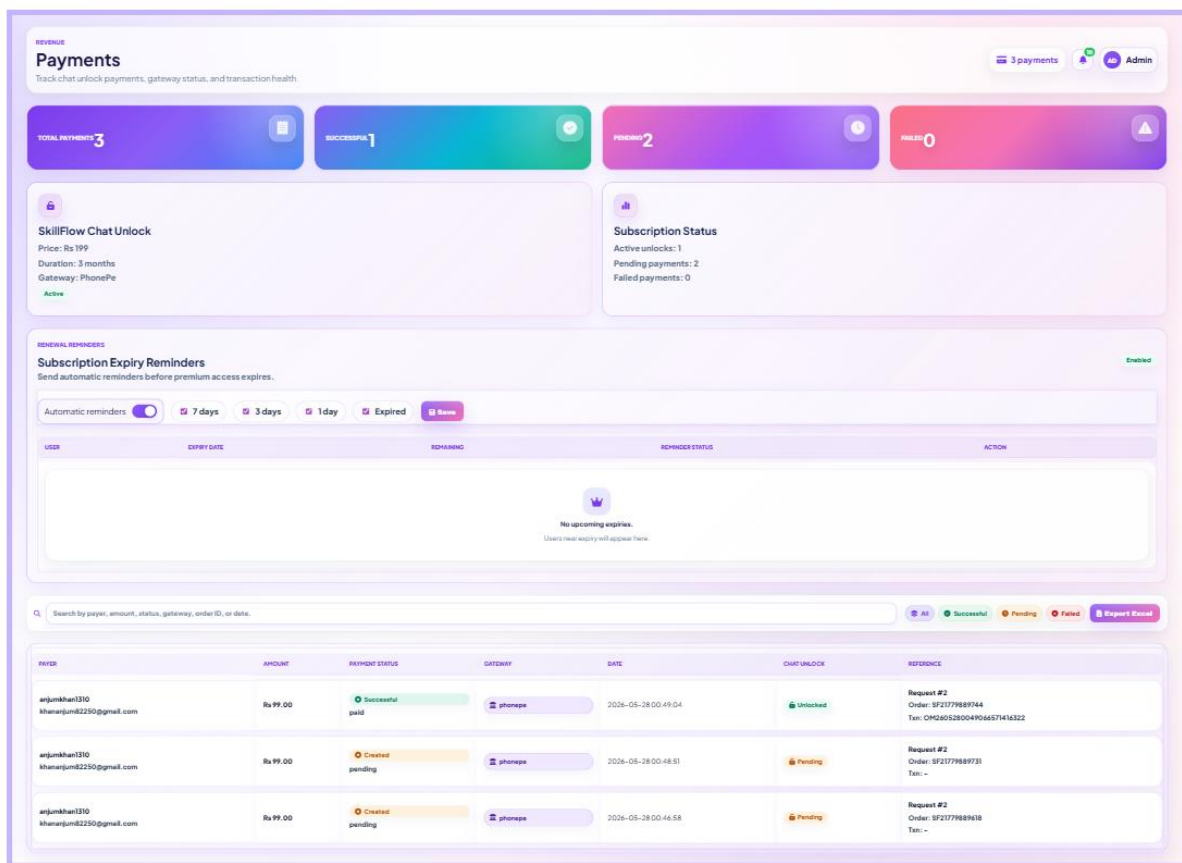


Figure 14.19: Admin Payments Page

The admin payments page tracks successful, pending and failed payment information. It is useful for payment monitoring and reporting.

14.20 ADMIN REPORTS PAGE

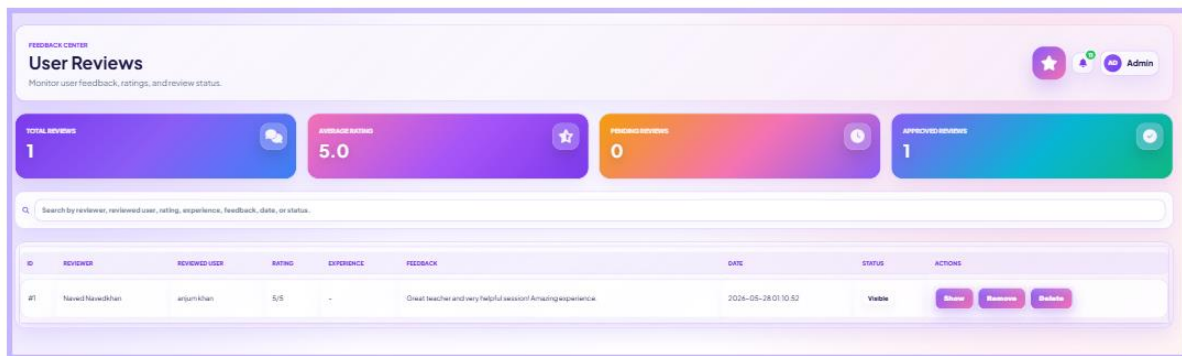


Figure 14.20: Admin Reports Page

The admin reports page presents user reports and moderation actions. It supports safer communication and accountable platform management.

14.21 ADMIN ACTIVITY LOGS PAGE

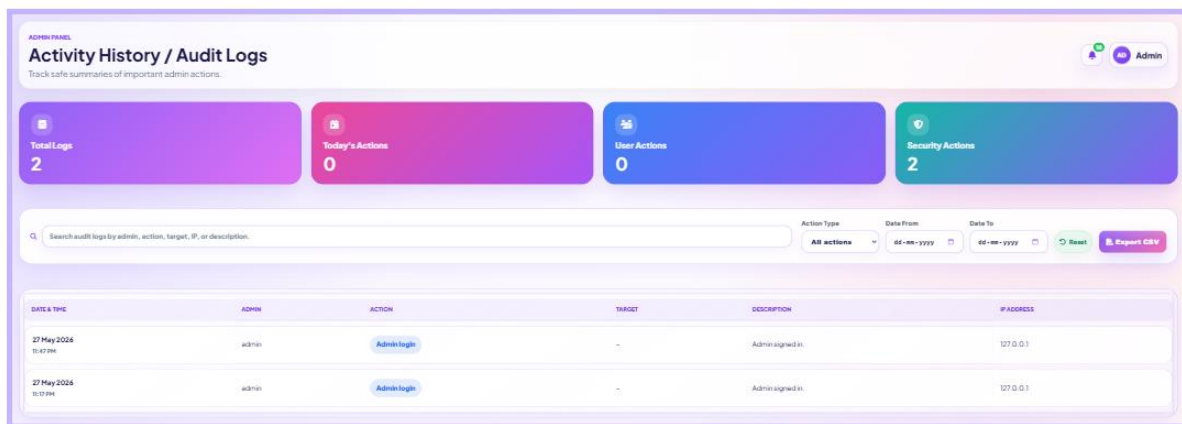


Figure 14.21: Admin Activity Logs Page

The admin activity logs page records important admin actions. It improves transparency and provides an audit trail.

14.22 MOBILE RESPONSIVE VIEW

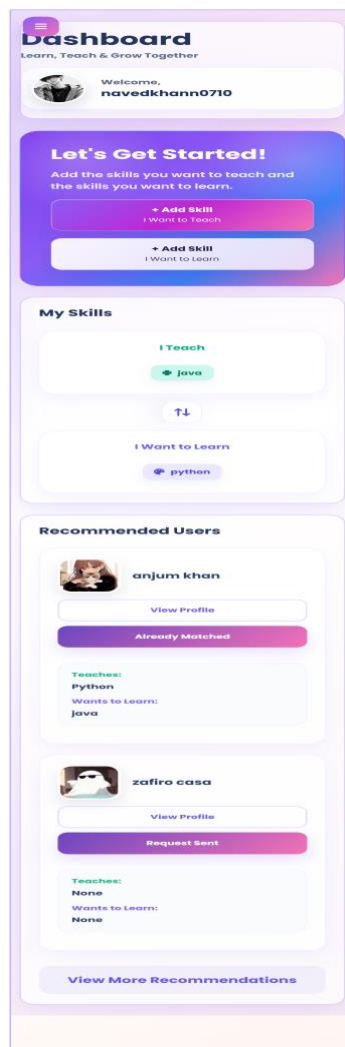


Figure 14.22: Mobile Responsive View

CHAPTER 15 - CONCLUSION

Project Achievements

Project Achievements in SkillFlow organizes requirement analysis around real users, admin responsibilities, security checks, database fields and the communication flow required for learning exchanges.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Area	Project-Specific Explanation
Learning	Learning is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Business value	Business value is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future readiness	Future readiness is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Business Impact

Business Impact in SkillFlow presents system design as the bridge between project idea and implementation, converting modules into architecture, diagrams, tables, pages and backend workflows.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Technical Learning

Technical Learning in SkillFlow describes the architecture through the browser, Flask application layer, route controllers, MySQL database and admin modules that cooperate during every transaction.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

User Benefits

User Benefits in SkillFlow explains data movement by showing how inputs from forms become requests, matches, messages, notifications, reviews, reports and analytics records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Admin Benefits

Admin Benefits in SkillFlow represents entity relationships among users, skills, requests, matches, payments, messages and reports so that database dependency can be understood clearly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Academic Outcome

Academic Outcome in SkillFlow documents user behaviour through UML style views that make login, profile editing, request sending, chat access and admin review easier to explain during viva.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

CHAPTER 16 - FUTURE SCOPE

AI Skill Recommendation

AI Skill Recommendation in SkillFlow describes database design with attention to field purpose, relationship meaning, integrity and the way MySQL supports persistent project records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Area	Project-Specific Explanation
Learning	Learning is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Business value	Business value is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future readiness	Future readiness is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Mobile Application

Mobile Application in SkillFlow explains each module by connecting the page seen by the user with the route, validation checks and table operations performed in the backend.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

AR Skill Preview

AR Skill Preview in SkillFlow presents authentication as the first controlled entry point where user identity, session values and validation rules protect the rest of the application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Voice Search

Voice Search in SkillFlow explains OTP verification as an additional confirmation step that improves trust before sensitive account actions are accepted by the system.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Chatbot

Chatbot in SkillFlow studies skill matching as the main business logic where offered skills, required skills and user interests are compared to suggest meaningful exchange opportunities.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Delivery Tracking

Delivery Tracking in SkillFlow explains the chat system as a controlled communication area that becomes useful after the system has enough information to connect two users responsibly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Multi Store Expansion

Multi Store Expansion in SkillFlow discusses payment integration through plan selection, transaction recording and administrative visibility rather than treating payment as a separate isolated feature.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Advanced Reports

Advanced Reports in SkillFlow describes the dashboard as the user's working area where personal details, skills, requests, matches, notifications and recent activity are brought together.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Better Database Constraints

Better Database Constraints in SkillFlow presents the admin panel as the management side of the software where platform activity can be supervised through users, reports, plans, analytics and notifications.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Progressive Web App Improvements

Progressive Web App Improvements in SkillFlow explains validation by showing how forms, route checks, database constraints and user feedback reduce incorrect entries and incomplete submissions.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

BIBLIOGRAPHY

Python Flask official documentation

MySQL documentation

Bootstrap documentation

MDN Web Docs

PhonePe gateway documentation

MCU BCA project guidance

BIBLIOGRAPHY Details

BIBLIOGRAPHY Details in SkillFlow discusses security from the viewpoint of sessions, password handling, OTP checks, role separation, controlled routes and careful processing of user supplied values.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Area	Project-Specific Explanation
Academic relevance	Academic relevance is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project relevance	Project relevance is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Usage	Usage is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

REFERENCES



SkillFlow project source files

SQL dump and Python Flask runtime DDL

local Flask/MySQL environment and Python FlaskMyAdmin workflow

Reference PDF used only for visual formatting study

REFERENCES Details

REFERENCES Details in SkillFlow explains testing through realistic operations such as registration, login, skill posting, request creation, chat unlock, payment recording and admin review.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

APPENDIX



Installation guide

SQL appendix

Source code appendix

APPENDIX Details

APPENDIX Details in SkillFlow presents results by connecting completed features with observable outputs on pages, database records and admin dashboard summaries.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

INSTALLATION GUIDE

Step	Instruction	Verification
1	Install local Flask/MySQL environment and start Apache/MySQL.	Complete
2	Copy SkillFlow inside project workspace.	Complete
3	Import database using Python FlaskMyAdmin.	Complete
4	Update environment configuration.	Complete
5	Open localhost:5000/ in browser.	Complete
6	Verify admin dashboard and user flows.	Complete

EXTENDED TECHNICAL APPENDIX

Prepared Statement Usage

Prepared Statement Usage in SkillFlow describes advantages through practical benefits such as organized exchange, reduced manual search, better accountability and improved learning collaboration.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Output Escaping Strategy

Output Escaping Strategy in SkillFlow discusses limitations honestly, including the dependency on correct user data, internet availability and future scope for stronger recommendation intelligence.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Session Fixation Prevention

Session Fixation Prevention in SkillFlow presents future scope by identifying improvements such as AI matching, mobile application support, advanced analytics, richer communication and scalable deployment.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Admin Authorization Checks

Admin Authorization Checks in SkillFlow concludes the project by summarizing how the planned learning exchange system was converted into a working Flask and MySQL based web application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Password Hashing Workflow

Password Hashing Workflow in SkillFlow introduces how the platform converts informal peer learning into an organized digital workflow where users can publish skills, discover compatible learners and begin structured exchanges through a Flask powered interface.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

OTP Expiry Workflow

OTP Expiry Workflow in SkillFlow explains the learning exchange idea from the user's point of view, showing how a person moves from registration to skill discovery, request creation, matching and controlled communication.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Payment Verification Audit

Payment Verification Audit in SkillFlow focuses on the practical need for a web based environment where students and professionals can trade knowledge without depending on offline contacts or scattered social media messages.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Request Status Life Cycle

Request Status Life Cycle in SkillFlow describes the project's core purpose: to bring skill listing, matching, request handling, chat access, notifications and administrative supervision into one connected application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Skill Image Path Management

Skill Image Path Management in SkillFlow presents the academic and technical goals of the project, especially the use of Python Flask, HTML, CSS, JavaScript and MySQL to solve a real collaboration problem.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Upload Folder Governance

Upload Folder Governance in SkillFlow defines the boundary of the application by separating learner features, admin controls, database operations, payment supported access and report handling into clear functional areas.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Category Navigation Workflow

Category Navigation Workflow in SkillFlow discusses the problem that learners often know what they want to learn but do not have a reliable way to find someone who needs their own skill in return.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Subcategory Mapping Workflow

Subcategory Mapping Workflow in SkillFlow shows how the development process was planned around requirement study, database design, route implementation, interface preparation, validation, testing and final deployment readiness.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Dashboard Editor Data Flow

Dashboard Editor Data Flow in SkillFlow explains how administrative automation supports the project by monitoring users, reports, requests, payments and platform activity from a controlled backend panel.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Hero Slider Content Flow

Hero Slider Content Flow in SkillFlow highlights the database driven nature of the system, where important actions are stored, retrieved and updated through MySQL tables instead of remaining temporary browser data.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Notification Read Tracking in SkillFlow connects digital learning requirements with practical software engineering by combining authentication, search, matching, messaging and analytics into a single student project.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Admin Login Activity Auditing

Admin Login Activity Auditing in SkillFlow emphasizes user convenience through clean navigation, responsive pages, visible status messages, quick notifications and a dashboard that keeps major actions easy to reach.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Report Query Aggregation

Report Query Aggregation in SkillFlow studies the existing limitations of manual learning exchange and explains why a dedicated platform gives better tracking, accountability and communication control.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Revenue Calculation Logic

Revenue Calculation Logic in SkillFlow presents the proposed solution as a manageable web application where every major process is represented through clear routes, templates, forms and database relationships.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Inventory Stock Observation

Inventory Stock Observation in SkillFlow examines technical feasibility through the availability of Flask, MySQL, browser based frontend tools and a development environment that can be maintained on standard systems.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Low Stock Decision Rule

Low Stock Decision Rule in SkillFlow evaluates operational feasibility by considering whether normal users can register, add skills, send requests, unlock chat and understand notifications without technical training.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

User Profile Data Flow

User Profile Data Flow in SkillFlow discusses economic feasibility because the project can be built with open source technologies while still supporting premium style features such as plans and payment records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Address Management Workflow

Address Management Workflow in SkillFlow organizes requirement analysis around real users, admin responsibilities, security checks, database fields and the communication flow required for learning exchanges.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Review Submission Workflow

Review Submission Workflow in SkillFlow presents system design as the bridge between project idea and implementation, converting modules into architecture, diagrams, tables, pages and backend workflows.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
--------------------	---

Return Request Workflow

Return Request Workflow in SkillFlow describes the architecture through the browser, Flask application layer, route controllers, MySQL database and admin modules that cooperate during every transaction.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Contact Enquiry Handling

Contact Enquiry Handling in SkillFlow explains data movement by showing how inputs from forms become requests, matches, messages, notifications, reviews, reports and analytics records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
--------------------	---

Payment Failure Handling

Payment Failure Handling in SkillFlow represents entity relationships among users, skills, requests, matches, payments, messages and reports so that database dependency can be understood clearly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Payment Pending Handling

Payment Pending Handling in SkillFlow documents user behaviour through UML style views that make login, profile editing, request sending, chat access and admin review easier to explain during viva.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Request Success Handling

Request Success Handling in SkillFlow describes database design with attention to field purpose, relationship meaning, integrity and the way MySQL supports persistent project records.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Search Relevance Flow

Search Relevance Flow in SkillFlow explains each module by connecting the page seen by the user with the route, validation checks and table operations performed in the backend.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Skill Filter Behaviour

Skill Filter Behaviour in SkillFlow presents authentication as the first controlled entry point where user identity, session values and validation rules protect the rest of the application.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Responsive Header Testing

Responsive Header Testing in SkillFlow explains OTP verification as an additional confirmation step that improves trust before sensitive account actions are accepted by the system.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Mobile Skill Grid Testing

Mobile Skill Grid Testing in SkillFlow studies skill matching as the main business logic where offered skills, required skills and user interests are compared to suggest meaningful exchange opportunities.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Chat Unlock Form Testing

Chat Unlock Form Testing in SkillFlow explains the chat system as a controlled communication area that becomes useful after the system has enough information to connect two users responsibly.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Admin Form Testing

Admin Form Testing in SkillFlow discusses payment integration through plan selection, transaction recording and administrative visibility rather than treating payment as a separate isolated feature.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

SQL Join Testing

SQL Join Testing in SkillFlow describes the dashboard as the user's working area where personal details, skills, requests, matches, notifications and recent activity are brought together.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Area	Project-Specific Explanation
Technical purpose	Technical purpose is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Project implementation	Project implementation is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Testing method	Testing method is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.
Future improvement	Future improvement is handled with practical skill exchange logic so that SkillFlow remains usable, testable and suitable for academic demonstration.

Database Backup Practice

Database Backup Practice in SkillFlow presents the admin panel as the management side of the software where platform activity can be supervised through users, reports, plans, analytics and notifications.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Deployment Environment Variables

Deployment Environment Variables in SkillFlow explains validation by showing how forms, route checks, database constraints and user feedback reduce incorrect entries and incomplete submissions.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Skillion Error Display Policy

Skillion Error Display Policy in SkillFlow discusses security from the viewpoint of sessions, password handling, OTP checks, role separation, controlled routes and careful processing of user supplied values.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

CSRF Token Flow

CSRF Token Flow in SkillFlow explains testing through realistic operations such as registration, login, skill posting, request creation, chat unlock, payment recording and admin review.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

Final Submission Checklist

Final Submission Checklist in SkillFlow presents results by connecting completed features with observable outputs on pages, database records and admin dashboard summaries.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

PROFESSIONAL PROJECT REVIEW NOTES

Final Database Consistency Review

Final Database Consistency Review in SkillFlow describes advantages through practical benefits such as organized exchange, reduced manual search, better accountability and improved learning collaboration.

The implementation uses frontend pages for interaction and backend Python Flask routes for validation, database work, redirects and status changes. This separation keeps the user interface clean while preserving control over sensitive operations.

For a BCA major project, this topic demonstrates analysis, design, implementation and testing. It also shows how a skill learning community can reduce manual dependency by using a database-driven website with admin automation.

Important quality concerns include data accuracy, security, responsive layout, readable code, reliable chat unlock behaviour and consistent purple-lavender SkillFlow branding. These concerns are reflected in module design and testing.

The workflow normally follows Frontend to Backend to Database to Response. Input is validated, stored or fetched using SQL, and returned to the user/admin through a controlled screen update or redirect.